Cloud Infrastructure and Architecture

M.Sc.(I.T.), VNSGU, Roohana Parabia and Anju Dhandhaniya

Topics Covered

1. Compute Services
2. Storage Services
3. Database Services
4. Network Services
5. Security Services

AWS

- **Amazon Web Services(AWS)** is a cloud service from Amazon, which provides services in the form of building blocks, these building blocks can be used to create and deploy any type of application in the cloud.

- These services or building blocks are designed to work with each other, and result in applications that are sophisticated and highly scalable.

- Amazon offers nearly 100 on-demand services and that list is growing daily.

Why is AWS so successful?

Why is AWS so successful?

Security and Durability — AWS encrypt the data, offering end-to-end privacy and storage.

Experience — Developers can rely on Amazon's established processes. Their tools, techniques and suggested best practices are built upon years of experience.

Flexibility — There is great flexibility in AWS, allowing developers to select the OS language and database.

Ease of Use — AWS is easy to use. Developers can swiftly deploy and host applications, build new applications or migrate existing applications.

Scalability — Applications can be easily scaled up or down depending on user requirements.

Cost Savings — Companies only pay for the computing power, storage and resources used, with no long-term commitments.

What are the Services provided by AWS ?

AWS finds applications in a variety of domains, and some of them are listed below:

Deployment and Management

Application Services

Foundation Services

Foundation Services of Cloud Providers

1. Compute Services
2. Storage Services
3. Database Services
4. Network Services
5. Security Services

The name and implementation method of services may differ depending on the cloud provider but all of them follow same basic concepts.

Compute Services in AWS

- In AWS, **Compute** refers to the processing power(CPUs, GPUs), memory(RAM), and networking resources required to run applications and process data. It is the "brain" of your cloud infrastructure, performing the actual calculations and executions that make your software work.

- AWS offers three primary models for compute, ranging from full control to high automation:
  - Virtual Machines (EC2)
  - Container Services (ECS and EKS)
  - Serverless (AWS Lambda)

Compute Services in AWS

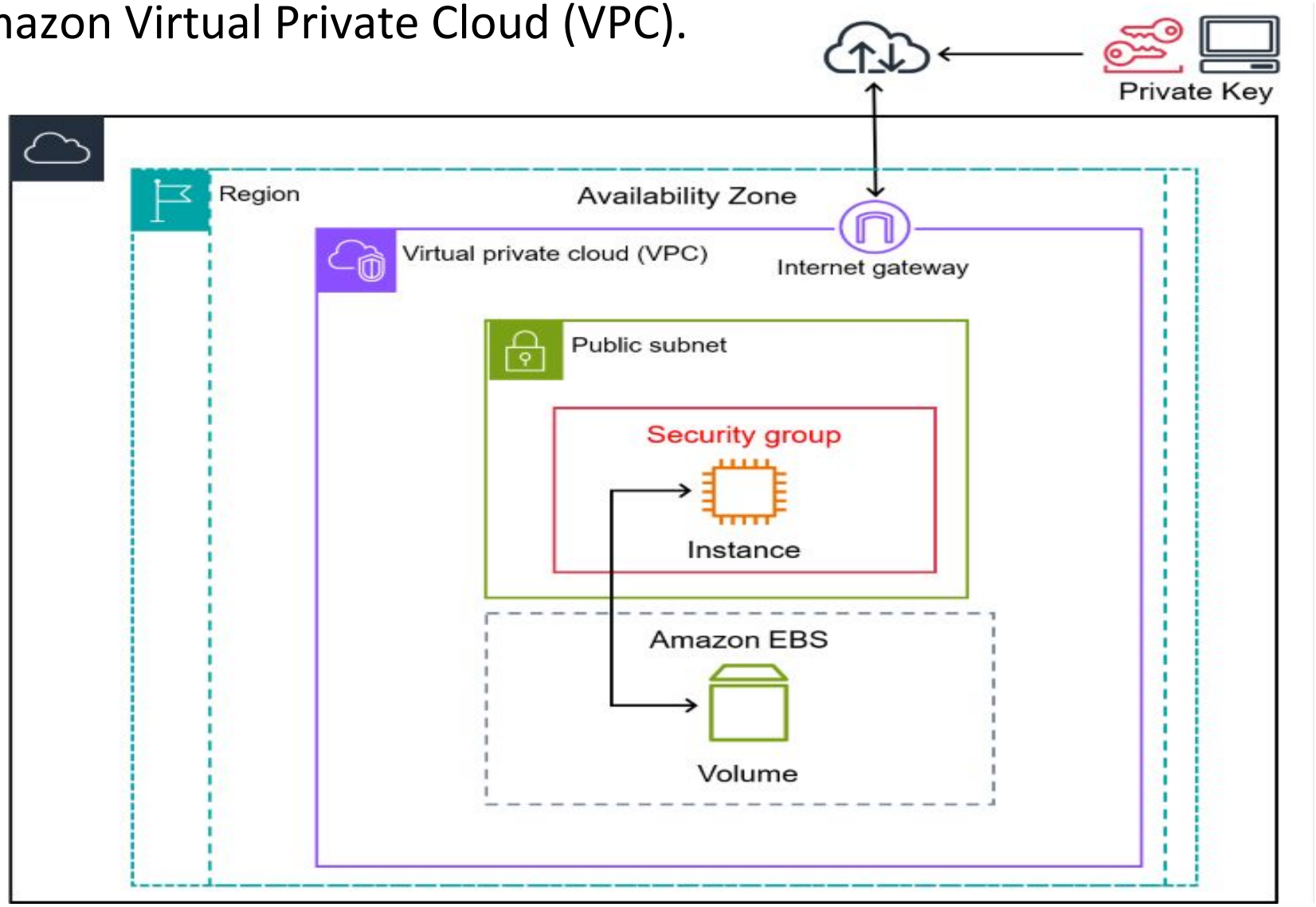
1. Virtual Machines (Amazon EC2)

Amazon EC2 provides virtual servers (called instances) that give you the highest level of control.

- **Control:** You choose the Operating System (Linux, Windows, etc.), CPU, RAM, and networking.
- **Management:** You are responsible for software patching, updates, and manual scaling.
- **Ideal for:** Traditional applications, legacy software that needs a specific OS, or complex workloads requiring full administrative access.

Compute Services (Amazon EC2)

- The following diagram shows a basic architecture of an Amazon EC2 instance deployed within an Amazon Virtual Private Cloud (VPC).



Compute Services (Amazon EC2)

- In this example, the EC2 instance is within an Availability Zone in the Region.
- The EC2 instance is secured with a security group, which is a virtual firewall that controls incoming and outgoing traffic.
- A private key is stored on the local computer and a public key is stored on the instance.
- Both keys are specified as a key pair to prove the identity of the user.
- In this scenario, the instance is backed by an Amazon EBS volume. The VPC communicates with the internet using an internet gateway.

Compute Services (Amazon EC2)

- To launch a new instance **click on the launch instance button.** This will open a wizard where you can **select the amazon machine image(AMI)** with which you want to launch the instance.
- Instance can be launched with a verity of operating system.
- When you launch an instance you **specify the instance type(micro, small, medium, large, extra-large, etc.),** the number of instances to launch based on availability zones for the instance.
- The instance launch wizard also allows you to specify the meta-data tags for the instance.
- When launching a new instance, the user selects a key-pair from existing keypairs or create a new keypair for the instance.
- Keypairs are used to securely connect to an instance after it launches.
- Security groups are used to open or block a specific network port for the launched instances.

Compute Services (Amazon EC2)

- **Steps to work with Amazon EC2:**

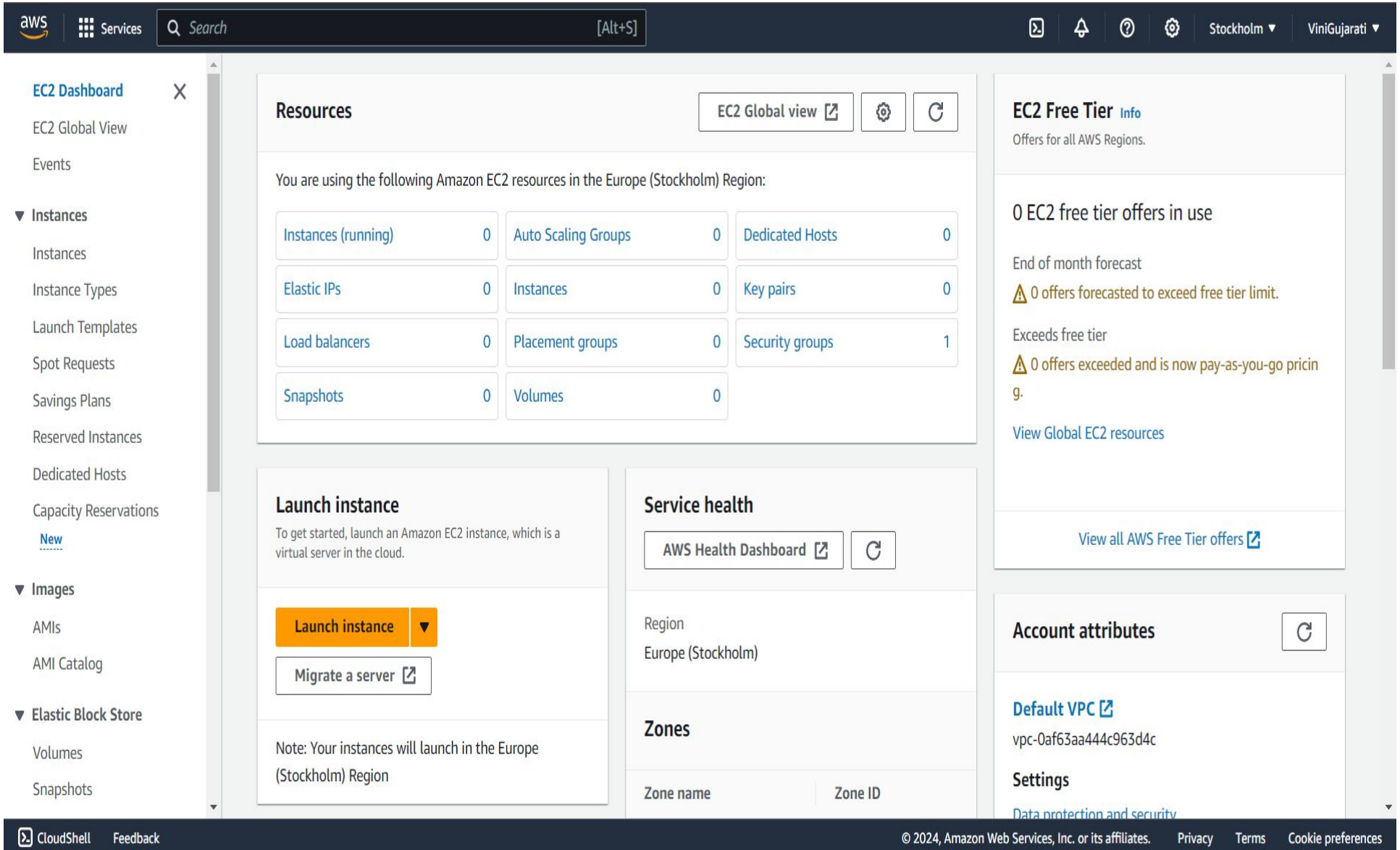
1. Sign in to the AWS Management Console.
2. Open the EC2 service from the dashboard
3. Launch an EC2 instance:

- Choose the AMI (Amazon Machine Image) for the instance, which is a pre-configured template with the necessary software.
- Select the instance type based on your requirements.
- Configure the instance details like network, subnet, security groups, and storage.
- Set up SSH key pairs for secure access to the instance.
- Review the instance details and launch it.

Compute Services (Amazon EC2)

4. Connect to the EC2 instance:
   - Obtain the public IP or DNS name of the instance.
   - Use SSH or Remote Desktop Protocol (RDP) to connect to Linux or Windows instances, respectively.
5. Configure and manage your instances:
   - Install and configure software on the instance.
   - Set up networking, security, and storage configurations.
   - Monitor and manage the instances using the EC2 management tools and AP
6. Terminate instances when they are no longer required to avoid unnecessary charges
   - These steps provide a high-level overview, and there are many additional configurations and features available in Amazon EC2.

Compute Services (Amazon EC2)



Compute Services in AWS

2. Container Services (ECS & EKS)

Containers bundle application code with its dependencies, ensuring it runs consistently across different environments.

- **Amazon ECS (Elastic Container Service):** A fully managed, AWS-native container orchestration service designed for simplicity.

- **Amazon EKS (Elastic Container Service for Kubernetes) :** A managed Kubernetes service for teams that need standard Kubernetes flexibility and portability.

Compute Services in AWS

3. Serverless (AWS Lambda) **Serverless computing** is a cloud model where you write and upload your code, and the cloud provider handles **all** the infrastructure. Despite the name, there are still servers involved, but they are "invisible" to you. You don't manage, patch, or scale them; you just focus on the code. **AWS Lambda** is the most famous example.

AWS Lambda allows you to run code without provisioning or managing any servers at all.

- **Event-Driven:** Code only runs when triggered by an event (e.g., a file upload to S3 or an API call).
- **Scaling:** It scales automatically from zero to thousands of requests per second.
- **Pricing:** You pay only for the milliseconds your code is running; there is no cost when it is idle.

Overview of Serverless Architecture

- **Functions** - Functions are small, discrete code units that perform a single task. A function requires compute resources like CPU and memory to run. The cloud provider allocates these resources only when required. It creates a temporary environment for the serverless function to run.
- **Certain events can *trigger* or make the code unit run**. For example, an event could run if a user selects a button in an application. The request would trigger a function that reads the database and returns the relevant information to the user.
- **Scaling requests -** The more requests a function receives, the more resources it needs to run. The serverless platform monitors the load and keeps allocating cloud resources to a near-infinite scale. A single serverless function can thus handle one or one million requests without code changes.
- Once a function stops receiving requests, the cloud provider takes down the associated infrastructure to save on cost. It allocates resources only when required. If there's no usage, the environment can scale to zero.

What are the types of Serverless Architecture?

In the world of **Serverless Computing**, **FaaS** and **BaaS** are the two building blocks that allow you to build apps without managing actual servers.

Function as a service (FaaS)

This is the **Logic** layer. You write a small block of code (a "function") that does one specific thing and upload it to the cloud.

- **How it works:** The code sits idle and costs **₹0** until an "event" (like a button click or a file upload) triggers it. It runs for a few seconds and then shuts down.

- **AWS Example:** AWS Lambda.

- **Analogy:** A **Vending Machine**. It stays off until you put a coin in, performs one specific task (gives you a soda), and then stops.

What are the types of Serverless Architecture?

Backend as a Service (BaaS)

This is the **Outsourced Features** layer. Instead of writing code for common tasks like user login, databases, or file storage, you use a ready-made service via an API.

- **How it works:** You "rent" the entire backend functionality. You don't write the logic; you just connect your frontend to it.

- **AWS Examples:** Amazon Cognito (for Login), Amazon DynamoDB (for Database), or Amazon S3 (for Storage).

- **Analogy:** A **Food Delivery App**. You don't cook the food (logic) or own the kitchen (server); you just order what you need, and it’s delivered ready-to-use.

-

What are the types of Serverless Architecture?

Feature	FaaS (Function)	BaaS (Backend)
Who writes the code?	**You** write the customer logic.	**The Provider.** You just use their API .
Purpose	To process data or run logic.	To store data or provide features.
Control	High (over the code itself).	Low (you use it “as it is”).
Scaling	Automatic per request.	Fully managed by the provider.
Main Use Case	Image resizing, API calls, etc.	Login, Database, etc.

What are the types of Serverless Architecture?

How They Work Together?

Most modern apps use both. For example, in a **Photo Sharing App**:

1. **BaaS (Amazon Cognito):** Handles the user login (you didn't write the login code).
2. **BaaS (Amazon S3):** Stores the high-resolution photo.
3. **FaaS (AWS Lambda):** Triggers automatically to resize that photo into a thumbnail.
4. **BaaS (Amazon DynamoDB):** Stores the link to the thumbnail in a database.

AWS Lambda (Serverless FaaS type)

- **AWS Lambda** is a compute service that lets **you run code without provisioning or managing servers.**

- Lambda runs your code on a high-availability compute infrastructure and **performs all of the administration of the compute resources, including server and operating system maintenance, capacity provisioning and automatic scaling, and logging.**

- With Lambda, all you **need to do is supply your code in one of the language runtimes that Lambda supports.**

- **You organize your code into Lambda functions. The Lambda service runs your function only when needed and scales automatically.**

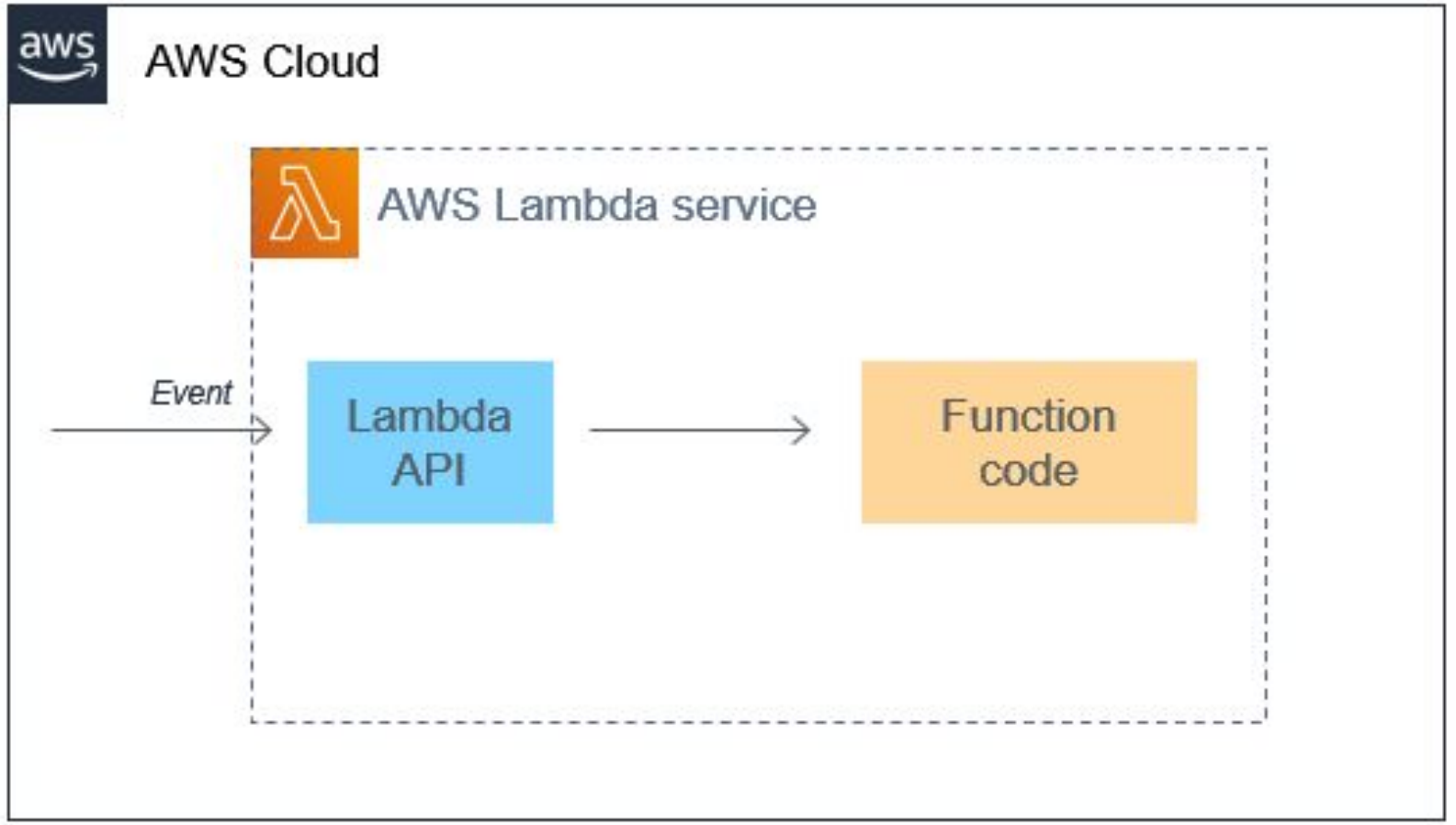
- You only pay for the compute time that you consume—**there is no charge when your code is not running.**

AWS Lambda

- **The main purpose of Lambda functions is to process events.**

- Most AWS services generate events, and many can act as an event source for Lambda. Within Lambda, your code is stored in a code deployment package.

- All interaction with the code occurs through the Lambda API and there is no direct invocation of functions from outside of the service.



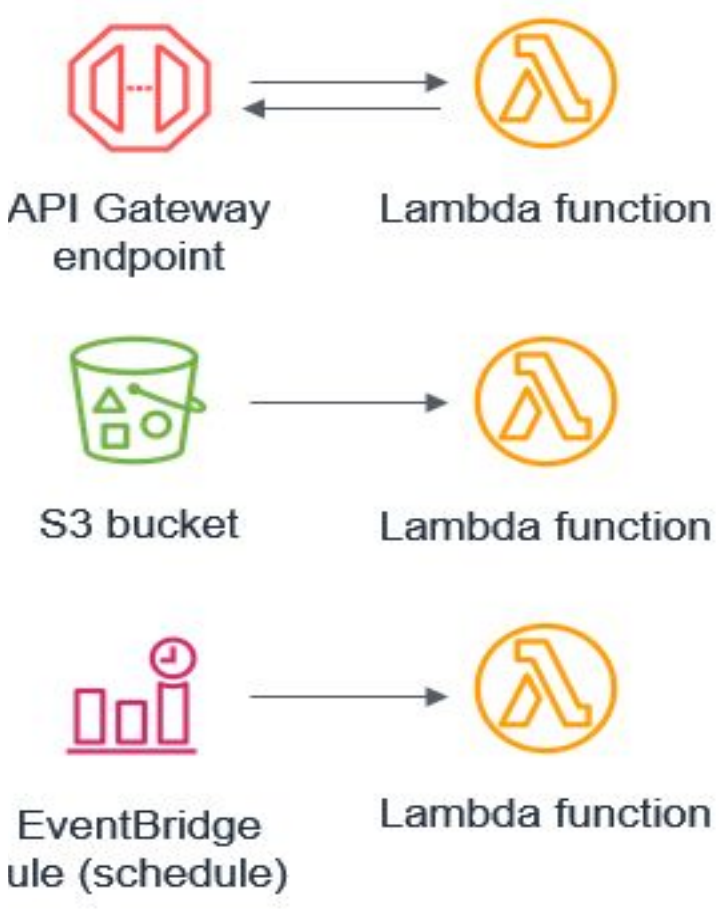
AWS Lambda

- An event triggering a Lambda function could be almost anything, from an HTTP request through API Gateway, a schedule managed by an EventBridge rule, an IOT event, or an S3 event. Even the smallest Lambda-based application uses at least one event.

- The event itself is a JSON object that contains information about what happened. Events are facts about a change in the system state, they are immutable, and the time when they happen is significant.

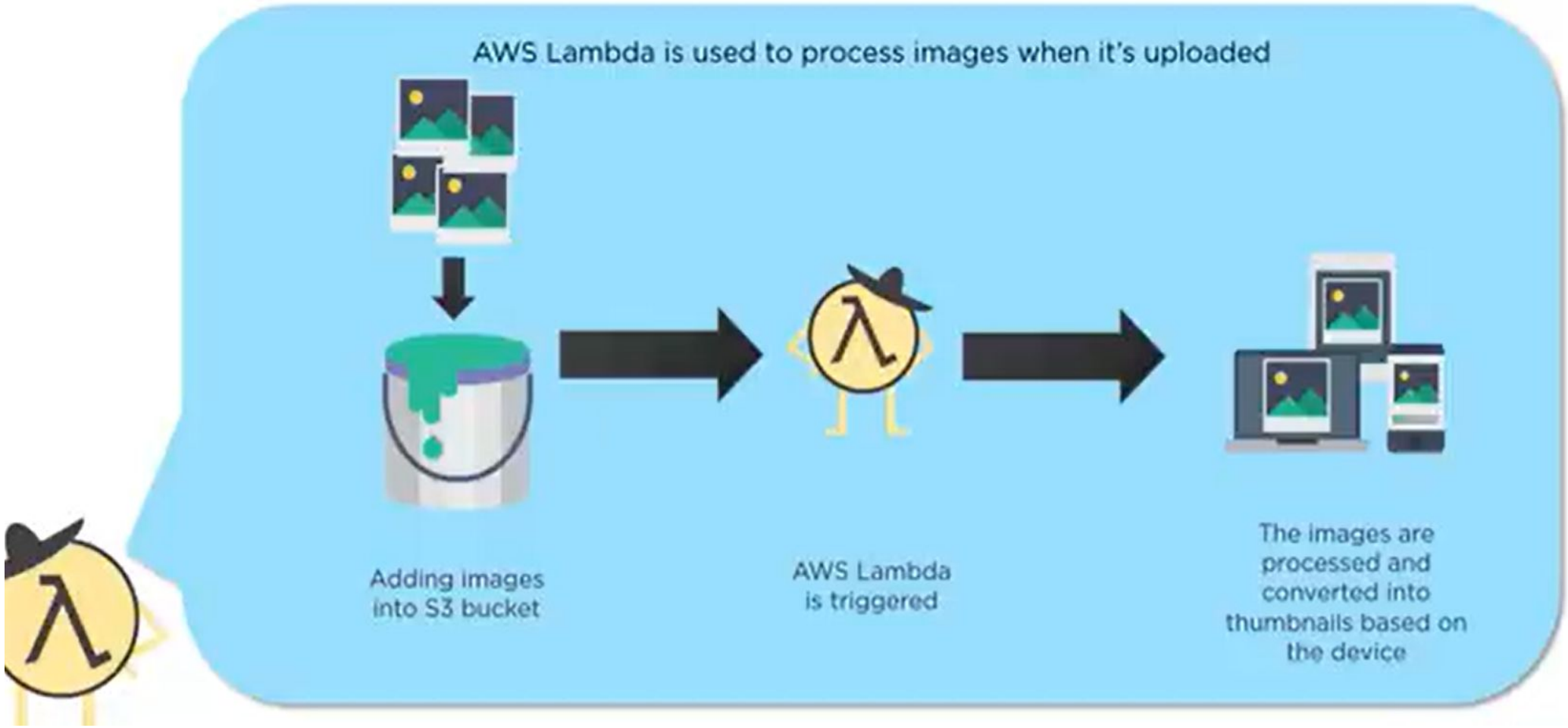
- The first parameter of every Lambda handler contains the event.

- An event could be custom-generated from another microservice, such as new order generated in an ecommerce application



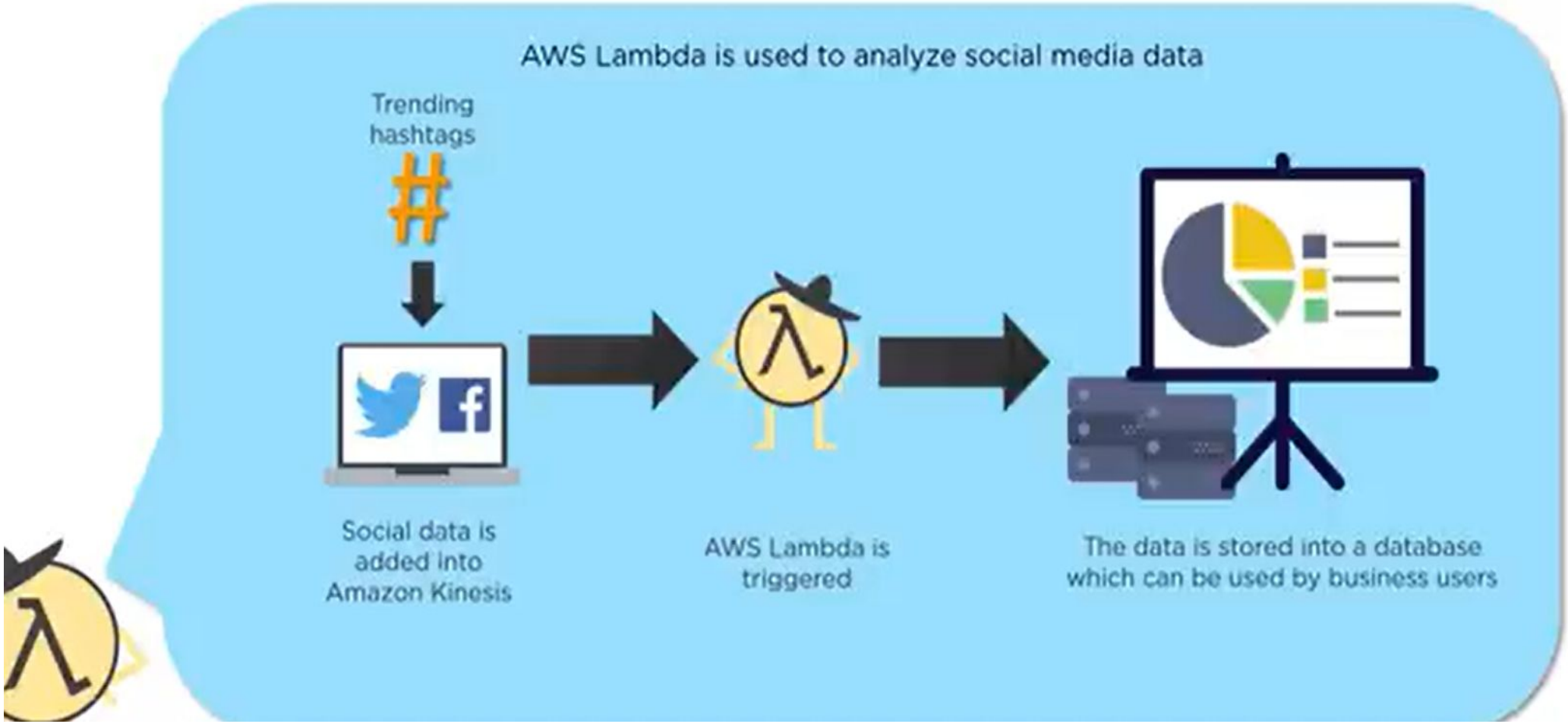
AWS Lambda

Where is Lambda used?



AWS Lambda

Where is Lambda used?



AWS App Runner (Serverless PaaS type/Managed Containers)

AWS App Runner is a fully managed service designed to simplify the deployment of containerized web applications and APIs. It allows developers to go from source code or a container image directly to a scalable, secure web service without managing any underlying infrastructure. You only manage your whole code rest everything is handled by AWS.

How it Works?

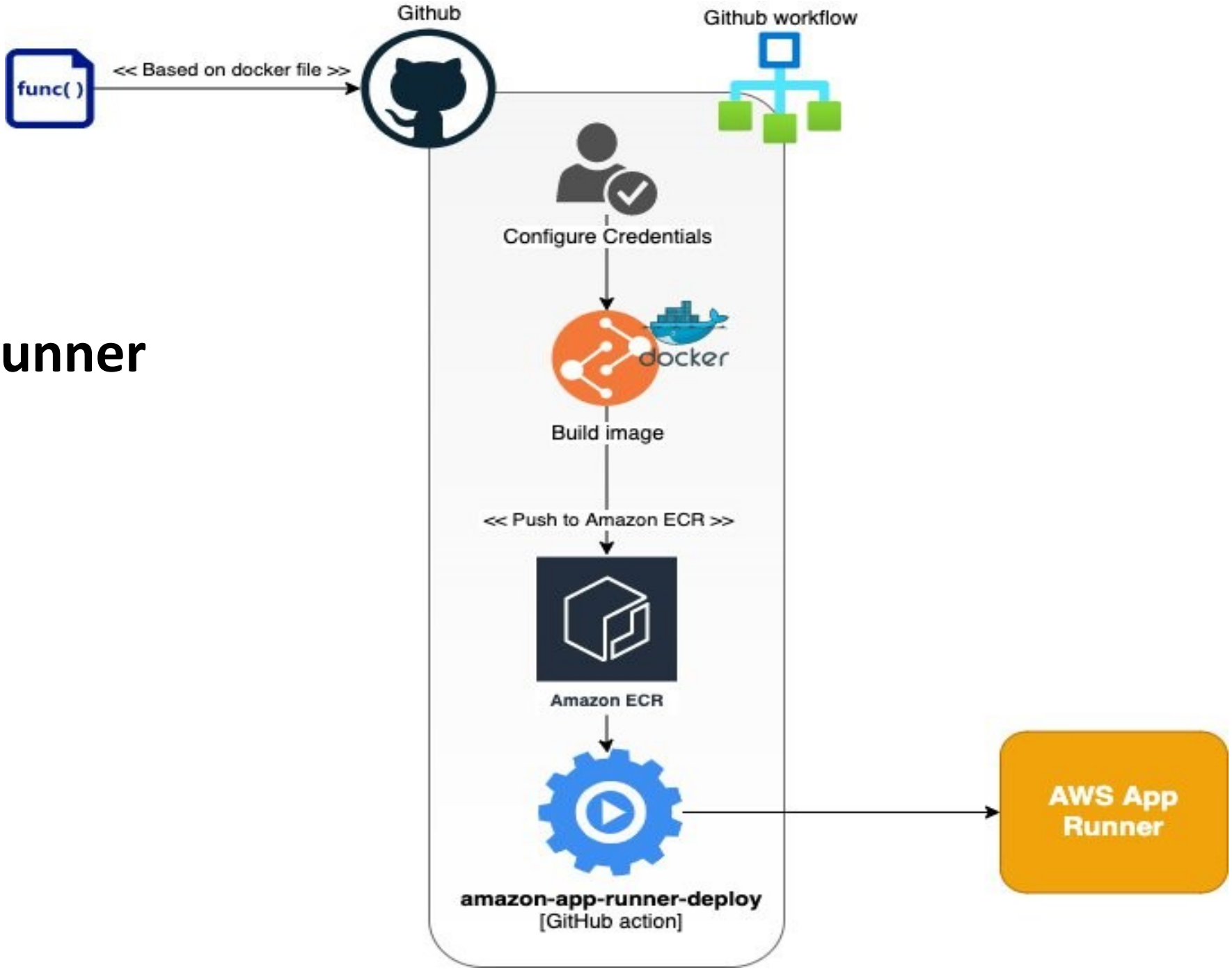
Behind the scenes, App Runner uses **Amazon ECS** and **AWS Fargate** to execute your containers. It abstracts these complex services so you don't have to configure task definitions, clusters, or VPC networking manually.

Use Cases:

Modern Web Apps: Ideal for backend web services, microservices, and frontend applications.

Rapid Prototyping: Perfect for startups or small teams that need to launch and iterate quickly without dedicated DevOps staff.

AWS App Runner Working



AWS AppRunner Working

Step 1: Prep Your Code

Ensure your project (Node.js, Python, etc.) has a file that tells it how to start (e.g., a package.json for Node or requirements.txt for Python). Push this code to a **Public or Private Repository** on GitHub.

Step 2: Start the Service

1.Log in to the **AWS App Runner Console**. 2. Click **Create service**. 3. For **Source**, select **Source code repository**.

4.Click **Connect to GitHub** and authorize AWS to "see" your account.

Step 3: Point to Your Repo

1.Select your **Repository** and the **Branch** (usually main). 2. Under **Deployment settings**, choose **Automatic**. (This means every time you "Push" to GitHub, your website updates instantly). Click **Next**.

Step 4: Configure the "Build"

1. **Runtime:** Pick your language (e.g., Python 3, Nodejs 18).
2. **Build command:** Enter how to install dependencies (e.g., npm install or pip install -r requirements.txt).
3. **Start command:** Enter how to run the app (e.g., node server.js or python app.py).
4. **Port:** Usually **80** or **3000** (Match whatever your code uses). Click **Next**.

Step 5: Name & Launch

1.Give your service a **Name** (e.g., my-first-web-app). 2. Choose the **CPU & Memory** (the smallest 0.25 vCPU is fine for testing). 3. Review everything and click **Create & Deploy**.

The Result: It takes about 2–5 minutes for AWS to pull the code, build the "container," and set up the networking. Once the status turns green (**Running**), AWS will give you a **Default Domain URL** (like https://abc123xyz.us-east-1.awsapprunner.com).

AWS App Runner (Serverless PaaS type/Managed Containers)

How is AWS App Runner different from ECS though both are Containers?

AWS App Runner (PaaS)

- App Runner is a managed container where whole infrastructure like VPC, Load balancing is handled by AWS. It is completely hidden from user. User has no control over infrastructure.
- Scaling is automatic based on HTTP request.
- It is deployed in minutes.
- Anybody who wants quick deployment without headache of networking can go for this option.

AWS ECS (Container Orchestration)

- Hear Infrastructure like VPC settings, load balancing, assigning CPUs and Memory, etc. is done by user. Have moderate control over Infrastructure.
- Setup requires deep configuration and proper skills so it may take hours or days.
- Use when app needs complex networking, and want to have full control over its infrastructure.

What is the difference between Lambda and AWS App Runner though both are serverless?

Feature	AWS Lambda	AWS App Runner
Code Style	**Snippet/Function.** (Specific logic only).	**Standard App.** (The whole source code).
Best for	Quick tasks triggered by events	Websites and APIs that stay “awake”
Connection Type	**Events**. Triggered by S3 database, API gateway	**HTTP/ Web**. It gives you a permanent URL
“Cold Start”	**Common**. The first request may be slow as it starts up.	**“Rare”**. It keeps a warm instance ready.
Execution time	**Max 15 Minutes.** Then it forces a shut down.	**Unlimited.** It stays running as long as you want.

Comparative Study

Feature	Virtual Machine (EC2)	Containers (ECS/EKS)	Managed Containers (App Runner)	Serverless (Lambda)
Control	Full (You manage OS, patches and security)	Moderate (You manage the app and its dependencies)	Low (You only manage the whole app code)	Low (You only manage the function code)
Scaling	Slow (Takes minutes to boot a whole OS)	Fast (Takes seconds to spin up a container)	Automatic	Instant (Scales automatically with every request)
Maintenance	High (You manage OS, network, updates)	Moderate(You manage clusters, Load Balancing, VPC)	None (AWS manages all)	None (AWS manages all)
Best for	“Always on” apps	Microservices	Microservices	Event – driven tasks
Cost Model	Pay by seconds (even if it is idle)	Pay by seconds (even if it idle)	Pay per second (when it is active)	Pay by request (only wile code is running)

Which Compute Service to Choose?

When to Use Which?

1. **Amazon EC2:** Best for **"Always On"** or heavy-duty apps. If you have a legacy Windows app or need a specific Linux kernel that requires "Root" access, go with EC2.
2. **Amazon ECS:** Best for **Microservices**. If you are building a modern app (like a food delivery or e-commerce site) where different teams manage different parts of the code, use ECS**.**
3. **AWS Lambda:** Best for **Event-Driven tasks**. If you need a script to run only when a user uploads a file, or to send a "Welcome" email, Lambda is the cheapest and easiest choice because it costs **₹0** when not in use.

The "Netflix" Example

- **EC2:** Runs their heavy, 24/7 internal databases.
- **ECS:** Runs their main API that handles your "Search" and "Login" requests.
- **Lambda:** Runs the script that generates movie thumbnails every time a new film is added.

Netflix follows a rule: If a task is **short, unpredictable, or event-driven**, they use **Serverless**. If the task is **constant and heavy** (like the actual streaming), they use **VMs/Containers**

Other Specialzed Compute Services

Other specialized compute services include:

- **Amazon Lightsail** for simple **Virtual Private Server (VPS)** hosting
- **Elastic Beanstalk(PaaS)** is an easy-to-use **Platform as a Service (PaaS)** that allows you to deploy and scale web applications without worrying about the underlying infrastructure.

What is a Cloud Storage?

Cloud storage provides a web service where your data can be stored, accessed and easily backed up by users over the internet

Note

Cloud storage is
- reliable,
- scalable and
- secure

than traditional on-premises storage systems

Data Storage Services in AWS

AWS provides a comprehensive range of storage services categorized by how data is accessed and used. Choosing the right one depends on whether you need to store files, raw data blocks for a server, or shared file systems.

Amazon S3

Durable object storage for all types of data

Amazon Glacier

Archival storage for infrequently accessed data

Amazon EBS

Block storage for use with Amazon EC2

Amazon EFS

File storage for use with Amazon EC2

Economic

Pay as you go

No upfront investment
No commitment

Easy to Use

Self service administration

SDKs for simple integration

Reduce risk

Durable and Secure

Avoid risks of physical media handling

Agility, Scale

Reduce time to market

Focus on your business, not your infrastructure

Data Storage Services in AWS

AWS categorizes its cloud storage into three primary architectural types, each designed for specific performance, accessibility, and cost requirements:

1. Object Storage

Designed for massive amounts of unstructured data. Instead of a file hierarchy, data is stored as "objects" in "buckets" with unique identifiers and metadata.

- **Amazon S3 (Simple Storage Service)**: It is built for storing and recovering any amount of data from anywhere over the internet. The primary choice for backups, data lakes, and static web hosting. It offers 11 nines (99.999999999%) of durability and 99.99% availability of objects.

- **Amazon S3 Glacier**: A low-cost sub-class of S3 optimized for long-term data archiving and "cold" storage, where data is rarely accessed.

Data Storage Services in AWS

2. Block Storage

Acts like a physical hard drive attached to a server. Data is split into fixed-size blocks, making it ideal for high-performance, low-latency applications.

- **Amazon EBS (Elastic Block Store)**: Persistent storage volumes used with **EC2 instances**. It is the standard choice for running databases (like MySQL or Oracle) and boot volumes.

- **EC2 Instance Store**: Temporary (ephemeral) block-level storage physically attached to the host computer. Data is lost if the instance is stopped or terminated.

Data Storage Services in AWS

3. File Storage

Provides a shared file system that can be accessed by multiple servers or instances simultaneously using standard protocols like **NFS** (for Linux) or **SMB** (for Windows).

- **Amazon EFS (Elastic File System):** A serverless, fully managed file system for Linux workloads that scales automatically.

- **Amazon FSx:** Specialized file systems for specific workloads, including **FSx for Windows File Server**, **FSx for Lustre** (for high-performance computing), and **FSx for NetApp ONTAP**.

Data Storage Services in AWS

Features	S3 (Object)	EBS (Block)	EFS (File)
Real World Analogy	A Warehouse	A Hard drive	A shared folder
Concurrent Access	Millions of users	Only 1 server at a time	Thousands of servers
Capacity	Virtually Infinite	Upto 16TB per volume	Scales automatically
Cost	Very Low	Moderate	High (but managed)

Data Storage Services in AWS

To see how these work together, imagine you are building a **scalable WordPress website** (like a high-traffic news site). In a modern AWS architecture, you wouldn't just pick one; you would use all three to handle different parts of the application:

The Architecture: A High-Traffic Website

1. EBS: The Server's "Brain"

- **Role:** The **Boot Drive** for your virtual servers (EC2).
- **Usage:** Each server needs an Operating System (Linux/Windows) and the web server software (Apache/Nginx) installed on it.
- **Why?** EBS provides the low-latency "block" speed needed to boot the server and run the software quickly.

Data Storage Services in AWS

2. EFS: The "Shared Library“

- **Role:** Shared **Configuration & Code** across multiple servers.
- **Usage:** In a busy site, you have 5 or 10 servers running at once to handle the traffic. All of them need to access the same WordPress /wp-content folder (plugins, themes, and configuration files).
- **Why?** If you update a plugin on Server 1, Server 5 needs to see that change immediately. EFS allows all servers to "mount" the same folder simultaneously.

3. S3: The "Infinite Media Vault“

- **Role: Images, Videos, and Backups**.
- **Usage:** Instead of keeping large high-resolution images on your servers, you upload them to an Amazon S3 bucket. When a user visits your site, their browser downloads the image directly from S3 (or via a CDN like Amazon CloudFront).
- **Why?** It’s the cheapest way to store petabytes of media, and it takes the load off your web servers.

Data Storage Services in AWS

How Netflix Uses AWS Storage

- **Amazon S3 (Object Storage): The Global Video Vault**
  - **Usage:** S3 is the "Source of Truth" for Netflix's entire content library.
  - **The Workflow:** When a new movie is finished, the high-quality master file is uploaded to an S3 bucket. From there, thousands of servers (EC2) pull that master file to convert it into the different resolutions (4K, 1080p, mobile) you see on your app. All those versions are stored back to another S3 Bucket.
    - When you hit "Play," you aren't actually pulling the file directly from the main S3 bucket (that would be slow if you're far away). Netflix uses a **Content Delivery Network** (CDN) called **Open Connect**. The files are copied from the **S3 bucket** to local servers "at the edge" (near your city) so the movie starts instantly without buffering.
  - **Archiving:** They use **S3 Glacier** to "freeze" older (cold data), less popular shows to save millions in storage costs.

Data Storage Services in AWS

- **Amazon EBS (Block Storage): The Engine's Hard Drive**
  - **Usage:** EBS provides the high-speed local storage for the thousands of **EC2 instances** (virtual servers) that power Netflix's backend.
  - **Performance:** When Netflix transcodes (converts) a 4K movie into 100 smaller files, it uses EBS for the "scratch space" because it needs extreme speed to write and read data while processing.

- **Amazon EFS / FSx (File Storage): The "Virtual Studio"**
  - **Usage:** Netflix uses shared file systems for its **Post-Production** and **Content Creation** teams.
  - **The Workflow:** Artists and editors working on the same original series need to share large media assets (like raw film clips) across multiple workstations. A shared file system allows several editors to "see" and edit the same project files at the same time, just like a shared office folder.

Data Storage Services in AWS

Netflix is estimated to spend approximately **₹727 crore to ₹1,136 crore per month** on AWS services.

Key Cost Drivers - Netflix's massive cloud expenditure is primarily driven by:

- **Video Processing (Transcoding):** Using **Amazon EC2** to convert high-quality master films into thousands of versions compatible with every device and internet speed globally.
- **Global Content Storage:** Storing petabytes of data across **Amazon S3** buckets, utilizing specialized classes like **Glacier Deep Archive** for long-term storage of older titles.
- **User Data Management:** Managing 325 million+ global subscribers using high-performance databases like **Amazon Aurora**.

Cost Optimization: Despite the high total, Netflix is highly efficient with its spending. They avoid even higher costs by building their own Content Delivery Network (**Open Connect**) to handle the actual streaming traffic to your home, which bypasses significant AWS data transfer fees. They also heavily use **AWS Spot Instances**—spare capacity offered at up to a 90% discount—for non-urgent tasks like video encoding.

How does EC2 Compute Service connect with Cloud Data Storage?

Think of EC2 as the **CPU/RAM (the computer)** and these storage services as the **different "drives"** you can plug into it.

1. The Main Pairing: EC2 + EBS (Block Storage)

This is the most common setup. Every EC2 instance needs an Amazon EBS (Elastic Block Store) volume to act as its **C: Drive** (Windows) or **Root Partition** (Linux).

- **What it holds:** The Operating System (Windows/Linux), your installed apps (Chrome, Python, Web Server), and your local app data.
- **Key Feature:** If you "stop" the EC2 instance, the EBS volume stays exactly as it is. It's persistent.

How does EC2 Compute Service connect with Cloud Data Storage?

2. The "Web" Pairing: EC2 + S3 (Object Storage)

EC2 doesn't "mount" S3 like a hard drive. Instead, your code running *inside* the EC2 instance uses the AWS SDK or CLI to "talk" to S3.

- **What it holds:** Large photos, video files, or backups.
- **The Workflow:** Your website on EC2 receives a user's photo and immediately "uploads" it to an S3 Bucket for safe keeping.

3. The Shared Pairing: EC2 + EFS / FSx (File Storage)

If you have 10 EC2 instances (a "fleet") and they all need to read/write to the same folder at the same time, you use Amazon EFS.

- **What it holds:** Shared media, code files, or user uploads.
- **Key Feature:** You "mount" it like a network drive (Z: drive) on all 10 servers simultaneously.

How does EC2 Compute Service connect with Cloud Data Storage?

If EC2 needs…	Connect to
To Boot the OS	Amazon EBS
A drive shared by many servers	Amazon EFS
Unlimited space for media files/ logs	Amazon S3

S3 Glacier almost always gets its input from an S3 Bucket, not directly from EC2.

S3 → S3 Glacier (Automatic): You set up an **S3 Lifecycle Policy**. This tells AWS: *"If a file is older than 30 days, move it automatically from S3 Standard to S3 Glacier."*

What is S3?

Amazon S3 (Simple Storage Service) provides object storage which is built for storing and recovering any amount of information or data from anywhere over the internet

What is S3?

General Storage Service

- Amazon Simple storage service(S3) is an online cloud-based data storage infrastructure for storing and retrieving any amount of data.
- S3 provides highly reliable, scalable, fast, fully redundant and affordable storage infrastructure.

How S3 Works (The 3 Essentials)

1. **Buckets (The Containers):** You create a "Bucket" (like a folder) and give it a globally unique name.
2. **Objects (The Files):** You upload your files into the bucket. Each file is called an "Object."
3. **Keys (The Address):** Every object has a unique "Key" (the file path and name) that allows you to find it instantly.

What is S3?

How S3 Works (The 3 Essentials)

1. **Buckets (The Containers):** You create a "Bucket" (like a folder) and give it a globally unique name.
2. **Objects (The Files):** You upload your files into the bucket. Each file is called an "Object."
3. **Keys (The Address):** Every object has a unique "Key" (the file path and name) that allows you to find it instantly.

Why Everyone Uses It

- **Durability (The "11 Nines"):** AWS designs S3 for **99.999999999% durability**. This means if you store 10 million objects, you might lose one every 10,000 years. It is much safer than any physical hard drive.
- **Scalability:** You can store a 1 KB text file or a 5 TB video. There is **no limit** to how much data a bucket can hold.
- **Static Web Hosting:** You can host an entire website (HTML, CSS, JS) directly from an S3 bucket without needing a server like EC2. Learn how to host a static website on S3.

What is S3?

Object & Bucket in Amazon S3

An object consists of data, key(assigned name) and metadata

A bucket stores objects

When data is added to the bucket, Amazon S3 creates a unique version ID and allocates it to the object

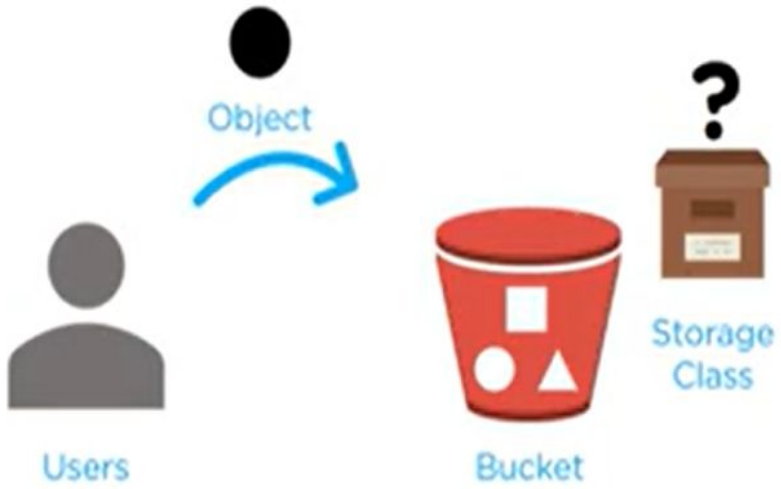
For Example:

Object: folder/Penguins.jpg → Key(name)
Bucket: simplilearn → Version ID
Link Address: https://s3.amazonaws.com/simplilearn/folder/Penguins.jpg

What is S3?

How does Amazon S3 work

- ✓ When files are uploaded to the bucket, the user will specify the type of S3 storage class to be used for those specific objects
- ✓ Later, users can define features to the bucket like bucket policy, lifecycle policies, versioning control etc.



Benefits of S3

Benefits of S3

S3 storage provides the following key features:

- **Buckets**—data is stored in buckets. Each bucket can store an unlimited amount of unstructured data.
- **Elastic scalability**—S3 has no storage limit. Individual objects can be up to 5TB in size.
- **Flexible data structure**—each object is identified using a unique key, and you can use metadata to flexibly organize data.
- **Downloading data—**easily share data with anyone inside or outside your organization and enable them to download data over the Internet.
- **Permissions—**assign permissions at the bucket or object level to ensure only authorized users can access data.
- **APIs** – the S3 API, provided both as REST and SOAP interfaces, has become an industry standard and is integrated with a large number of existing tools.

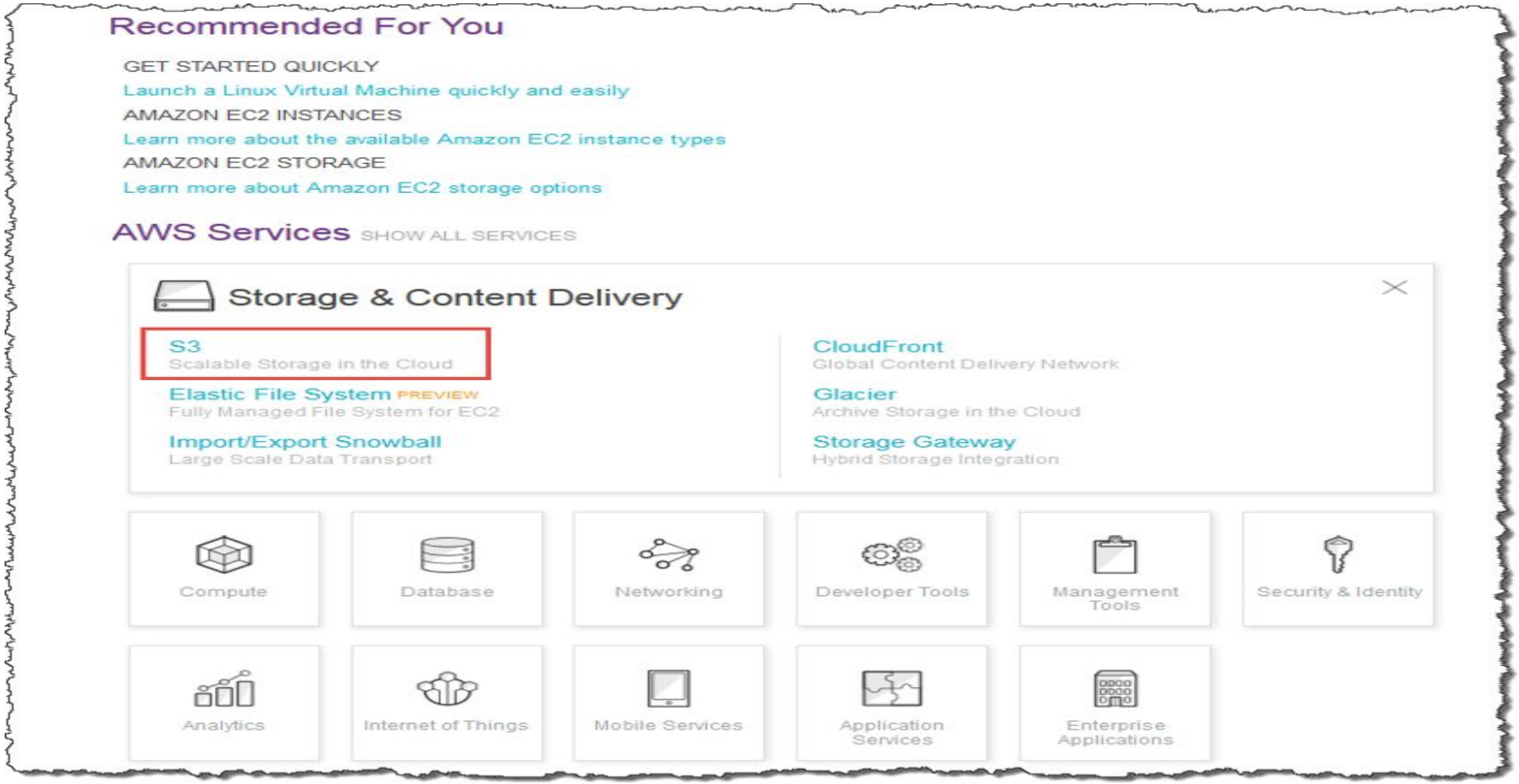
How to create S3 bucket?

Getting Started with Amazon S3

Here is a quick guide showing how to start saving your data to S3.
Create S3 Bucket - To get started with Amazon S3, the first step is to create an S3 bucket. A bucket is a container for storing objects in S3. **Follow these steps to create an S3 bucket:**

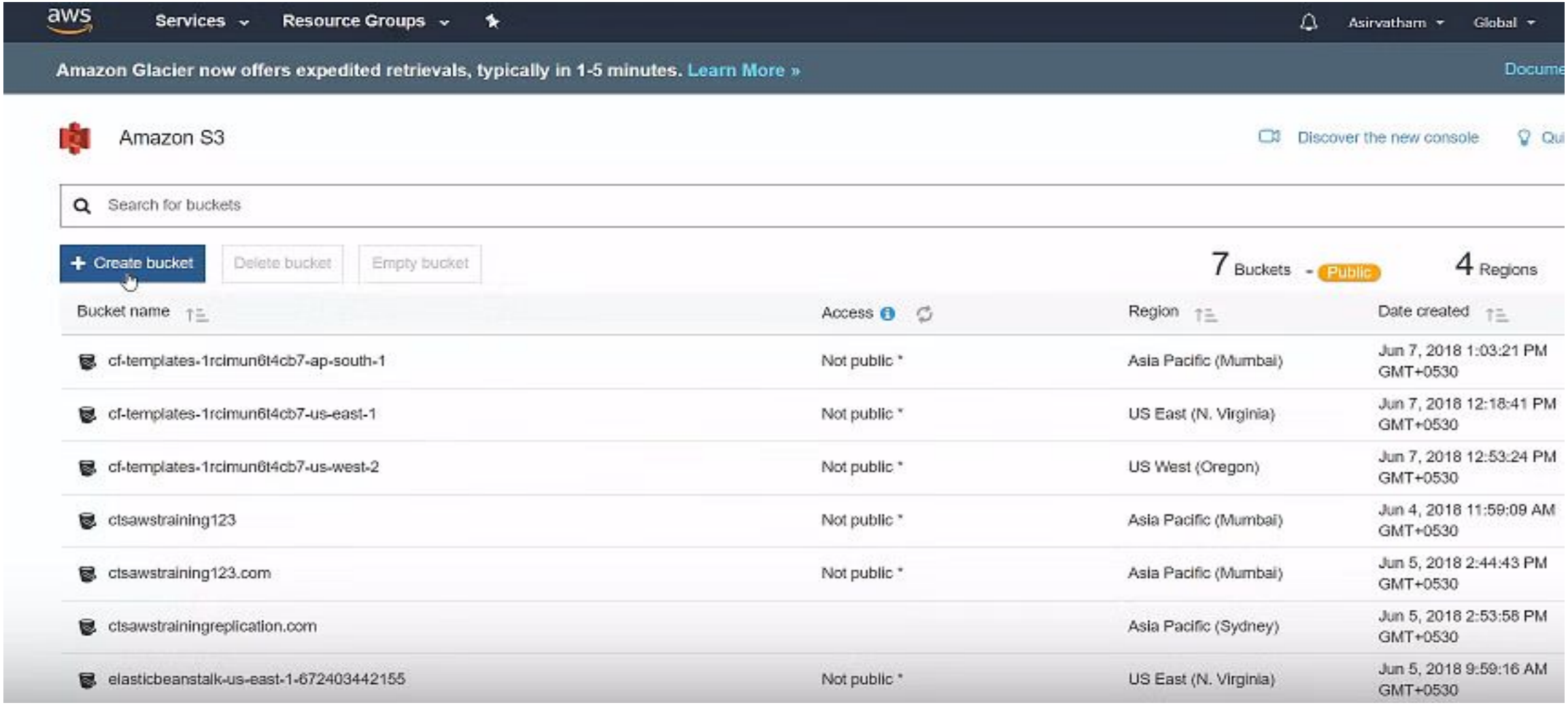
1. Log in to the AWS Management Console and navigate to the S3 service. Click the **Create bucket** button.
2. Enter a unique bucket name. Bucket names must be globally unique across all AWS accounts.
3. Choose the region where you want to create the bucket. The region selection is important because it affects the data transfer costs and latency.
4. Configure additional settings such as versioning, logging, and tags if needed.
5. Click on the **Create bucket** button to create your S3 bucket.
6. Once the bucket is created, you can start uploading objects to it.

How to create S3 bucket?



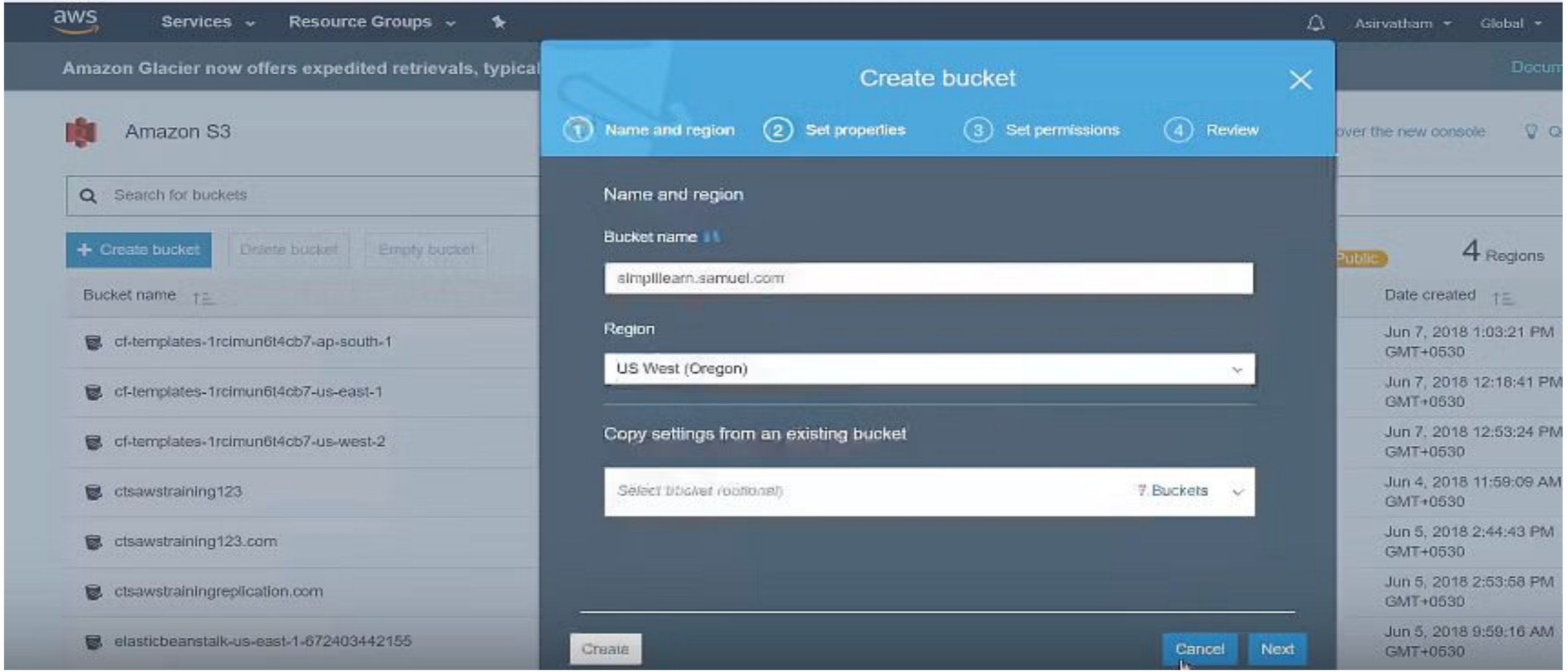
How to create S3 bucket?

Amazon S3 bucket list (usually empty for first-time users); create a bucket by clicking on the “Create bucket” button



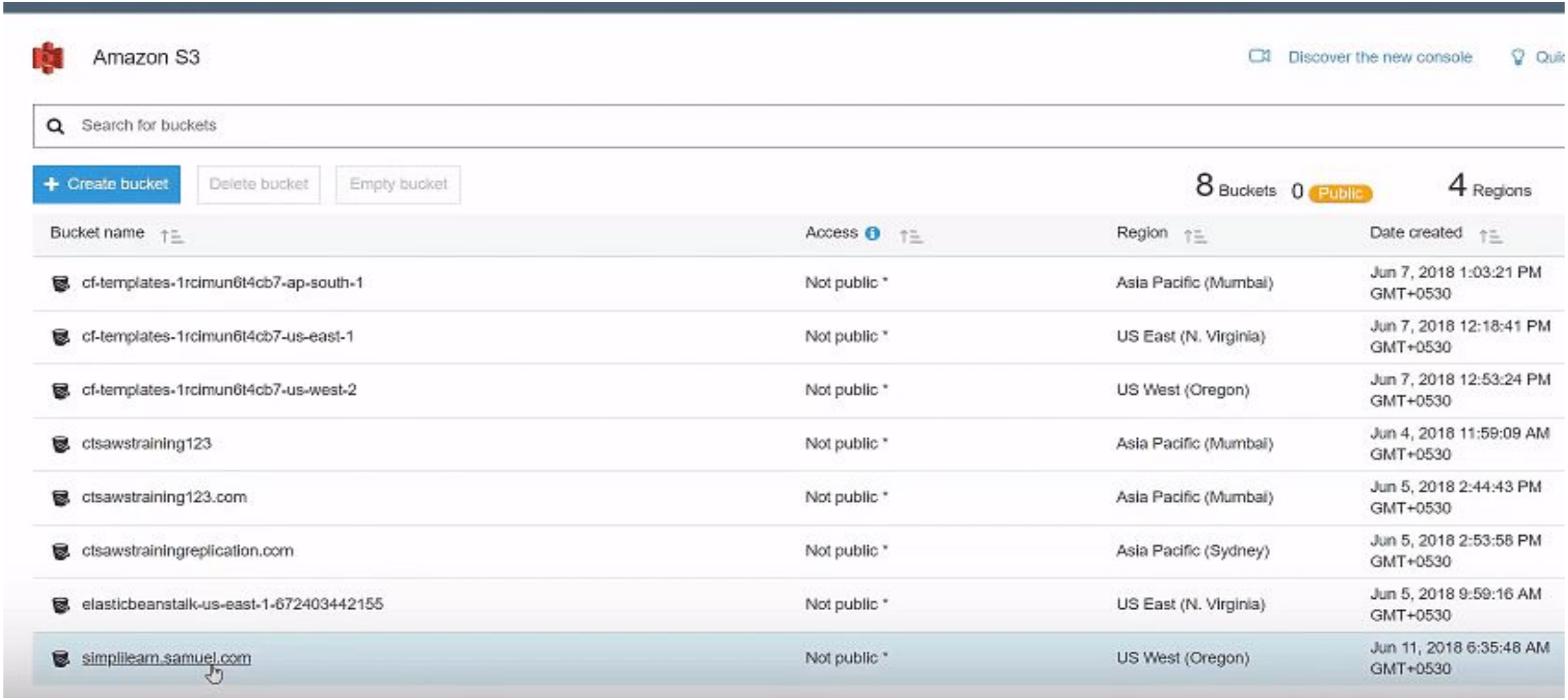
How to create S3 bucket?

Create a bucket by setting up name, region, and other options; finish off the process by pressing the “Create” button



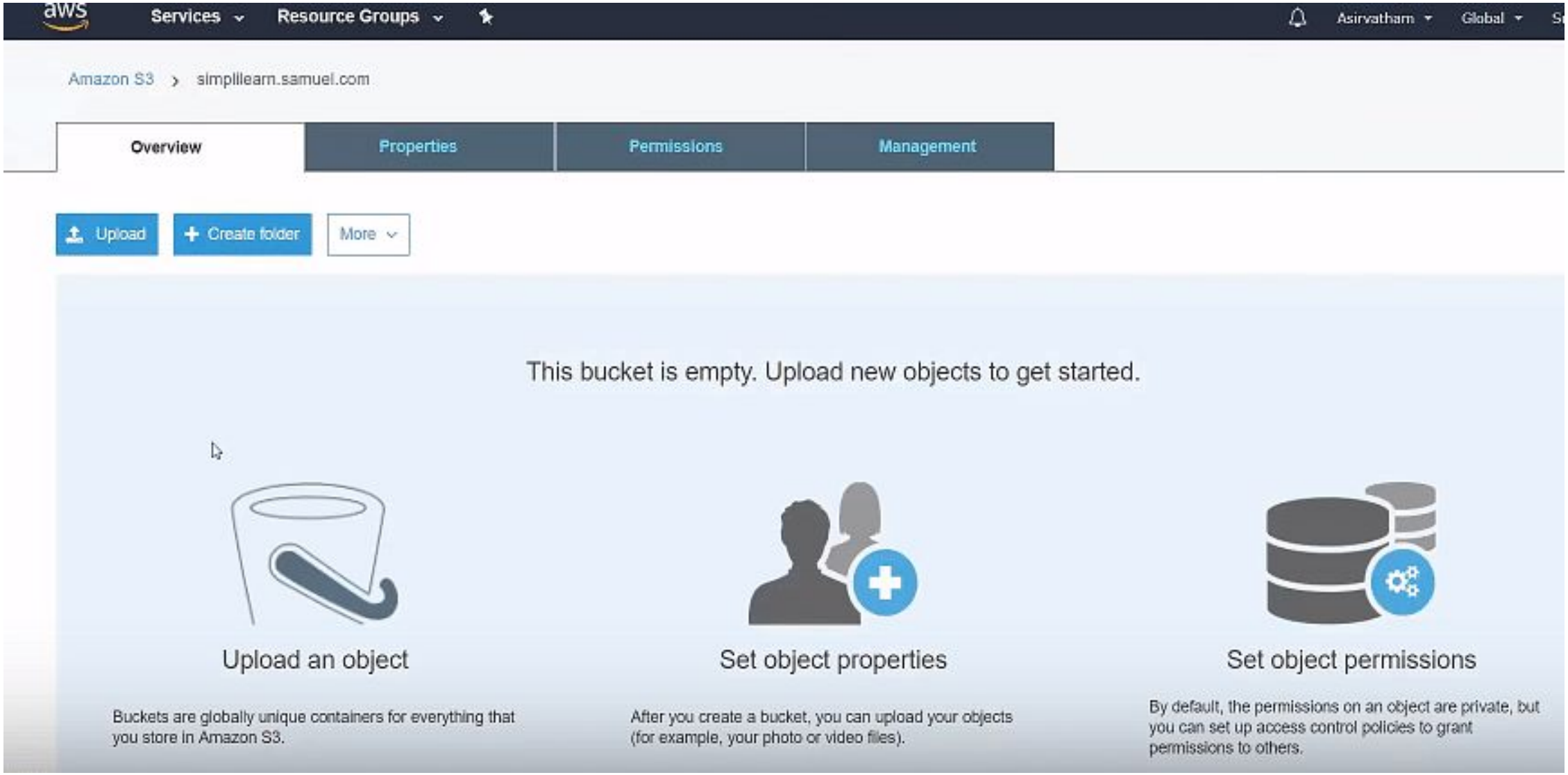
How to create S3 bucket?

Select the created bucket



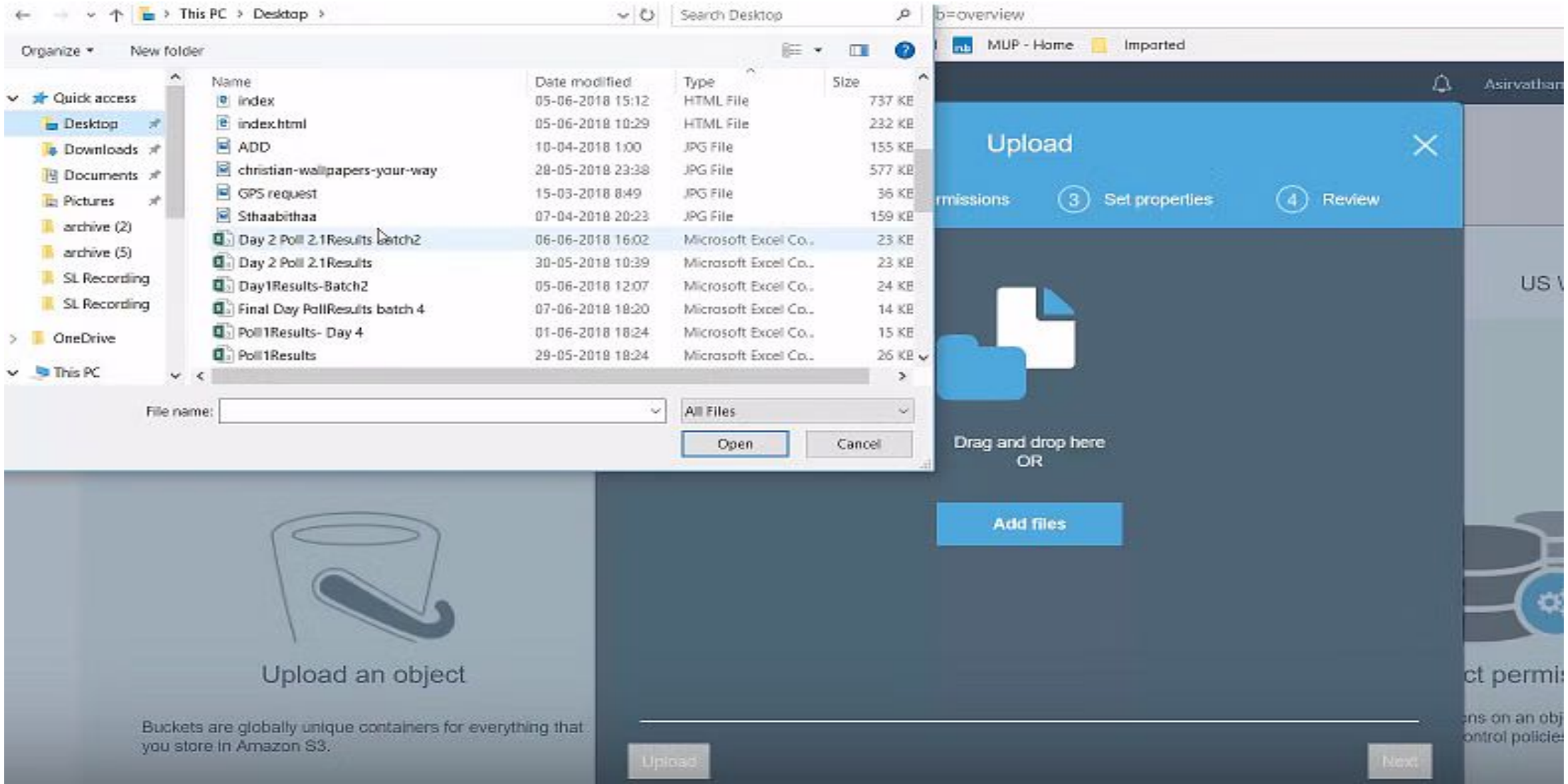
How to create S3 bucket?

Click on upload to select a file to be added to the bucket



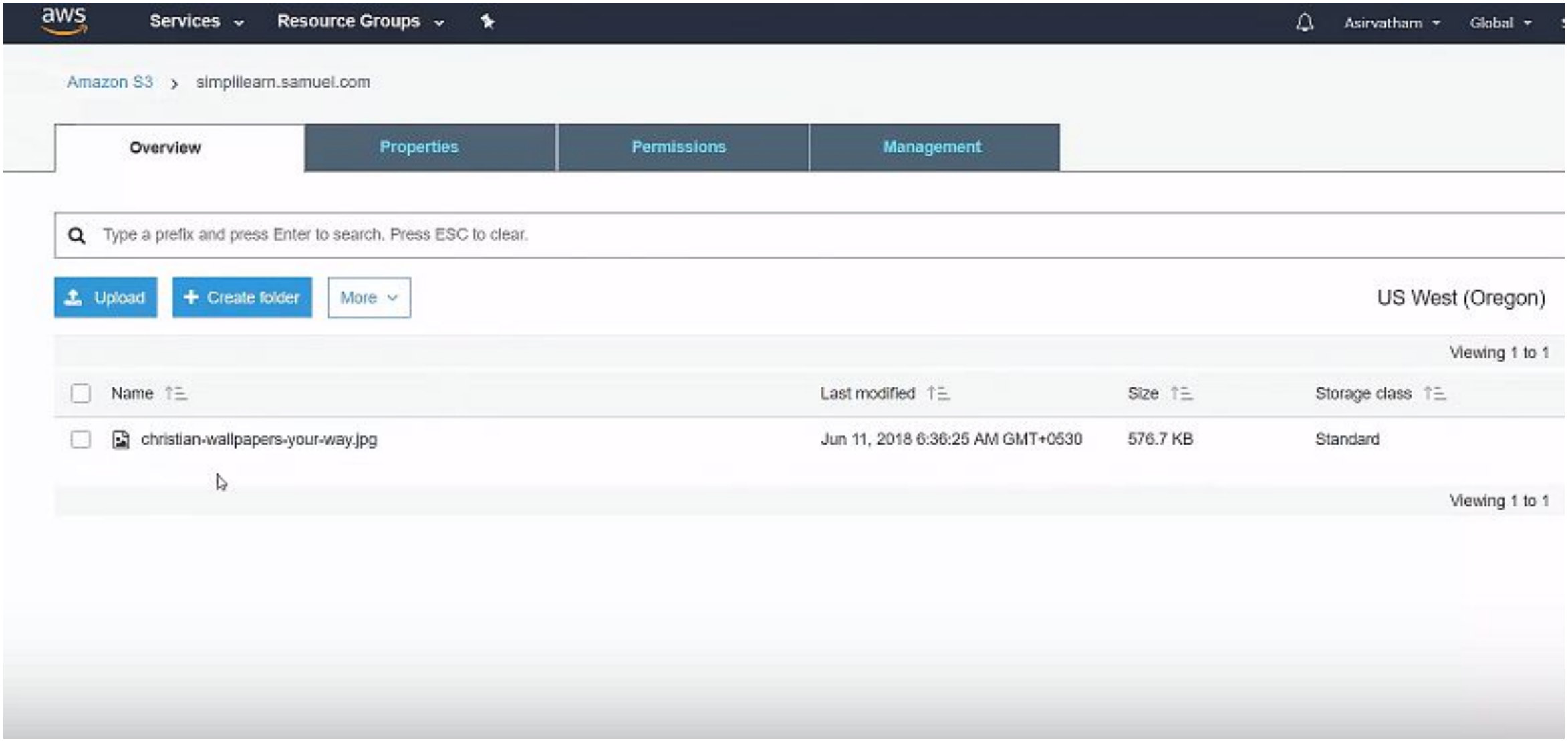
How to create S3 bucket?

Select a file to be added



How to create S3 bucket?

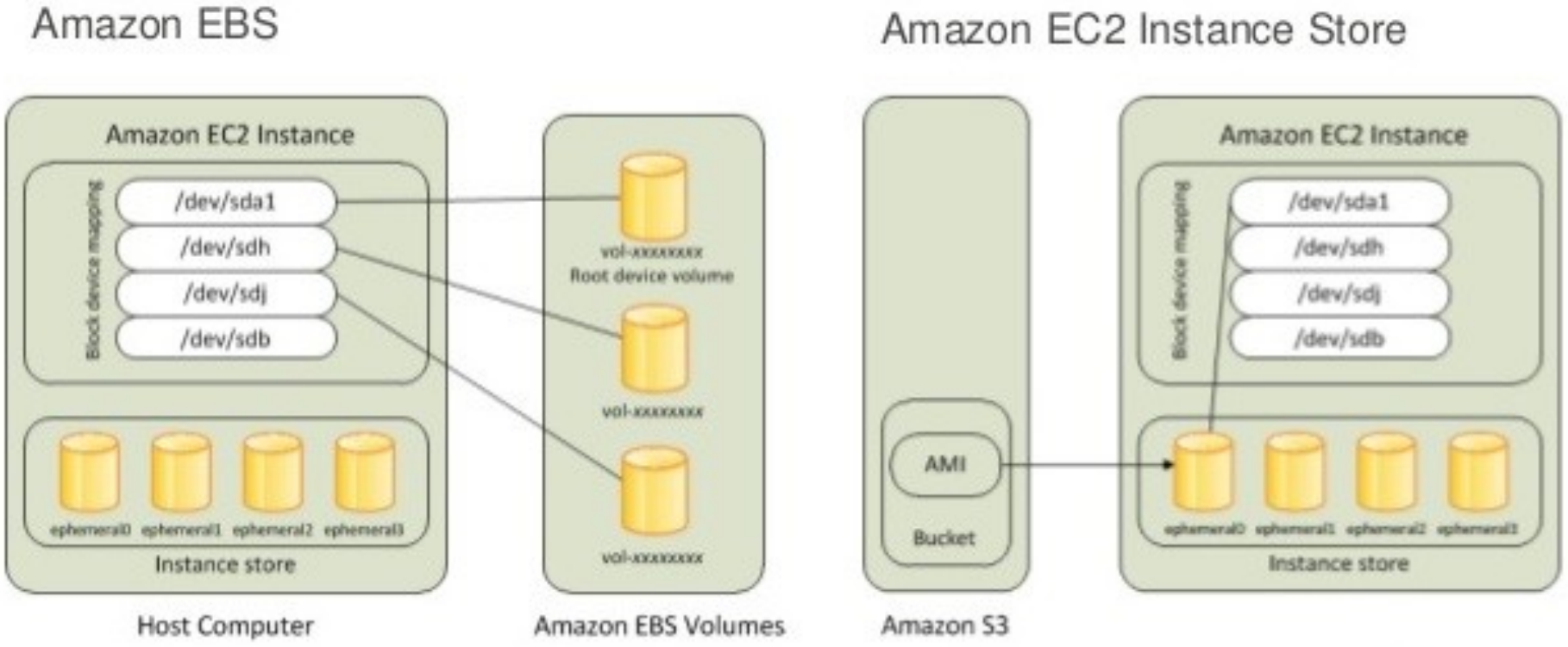
The file is now uploaded into the bucket



Elastic Block Service

Amazon Elastic Block Store (EBS) is a high-performance, persistent block storage service designed to act as the primary hard drive for Amazon EC2 instances. Unlike the temporary "Instance Store," EBS volumes retain their data even after you stop or reboot your server.

Network vs. Physical: EBS (left) is network-attached storage that persists independently of the server. Instance Store (right) is physically attached to the host hardware and is ephemeral—data is lost if the instance stops.



Elastic Block Service

Why you need EBS

1. **Persistence:** If your EC2 server crashes or you turn it off, your data stays safe on the EBS volume. When you start a new server, you just "plug" the old EBS volume back in. Learn about EBS persistence.

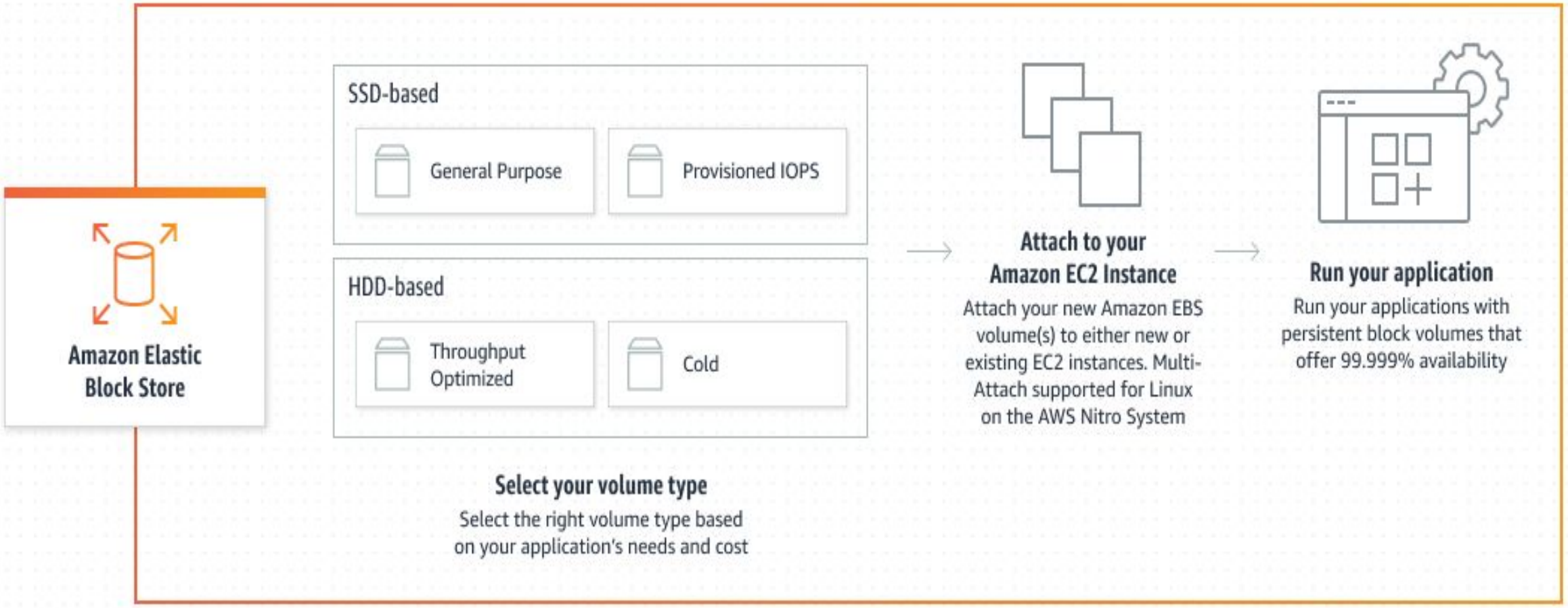
2. **High Speed (Low Latency):** Because it is "block storage," it handles data in tiny chunks very quickly. This is essential for **Databases** (like MySQL) that need to read/write thousands of times per second.

3. **Snapshots:** You can take a "picture" (Snapshot) of your entire drive at any moment. AWS stores these snapshots in **S3** for extra safety.

Elastic Block Service

There are several EBS volume types under these categories

- Solid State Drives (SSD)
- Hard Disk Drives (HDD).



Elastic Block Service

Architecture & Safety

The Availability Zone Lock

An EBS volume is replicated within a single **Availability Zone (AZ)** to prevent data loss from hardware failure.

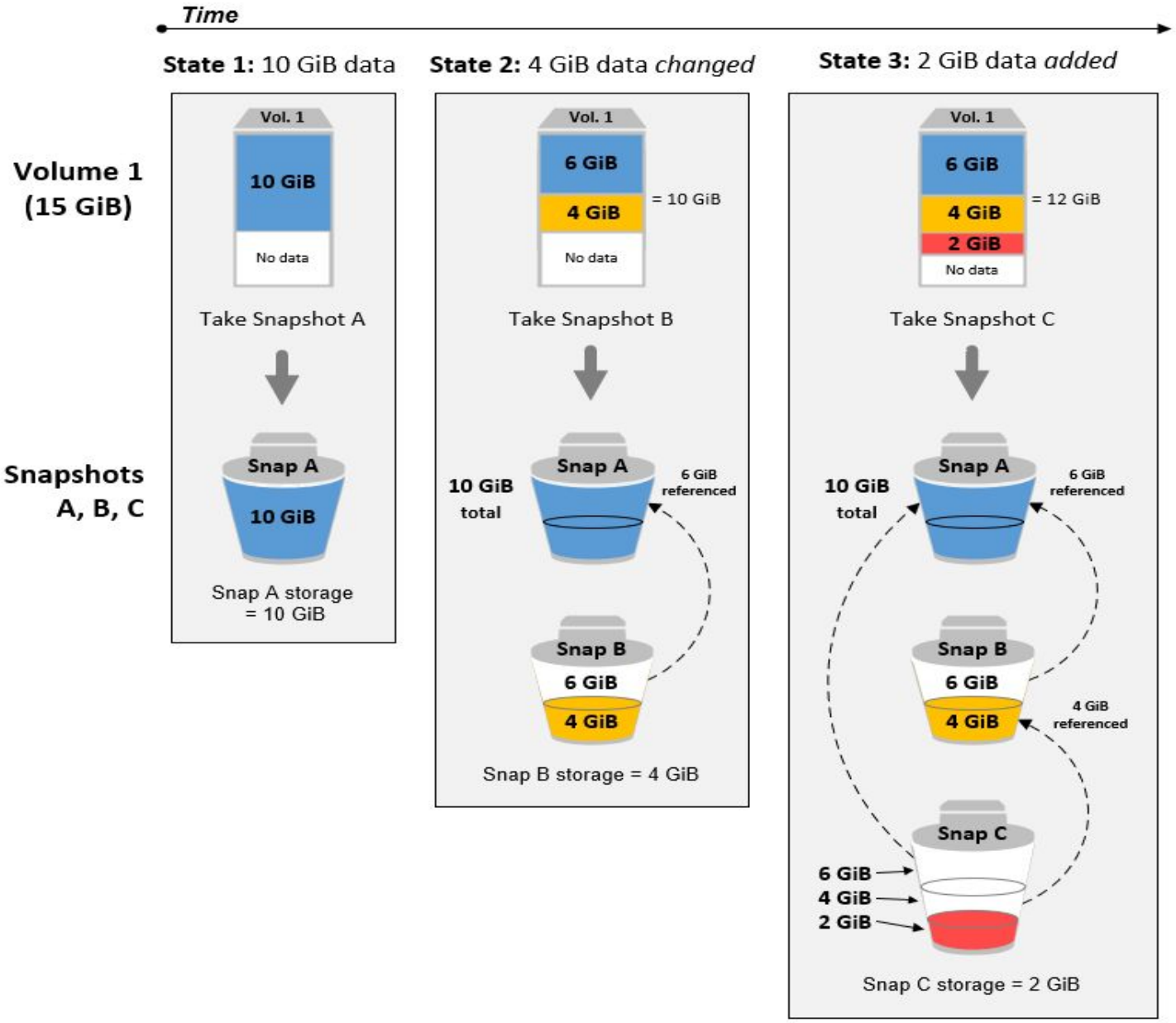
- **Constraint:** You can only attach a volume to an EC2 instance in the same AZ (e.g., us-east-1a).
- **Migration:** To move data to another AZ, you must create a **Snapshot**, then restore that snapshot to a new volume in the target zone.

Incremental Snapshots

EBS Snapshots are point-in-time backups stored in **Amazon S3**. They are **incremental**, meaning you are only billed for the blocks of data that have changed since the last snapshot.

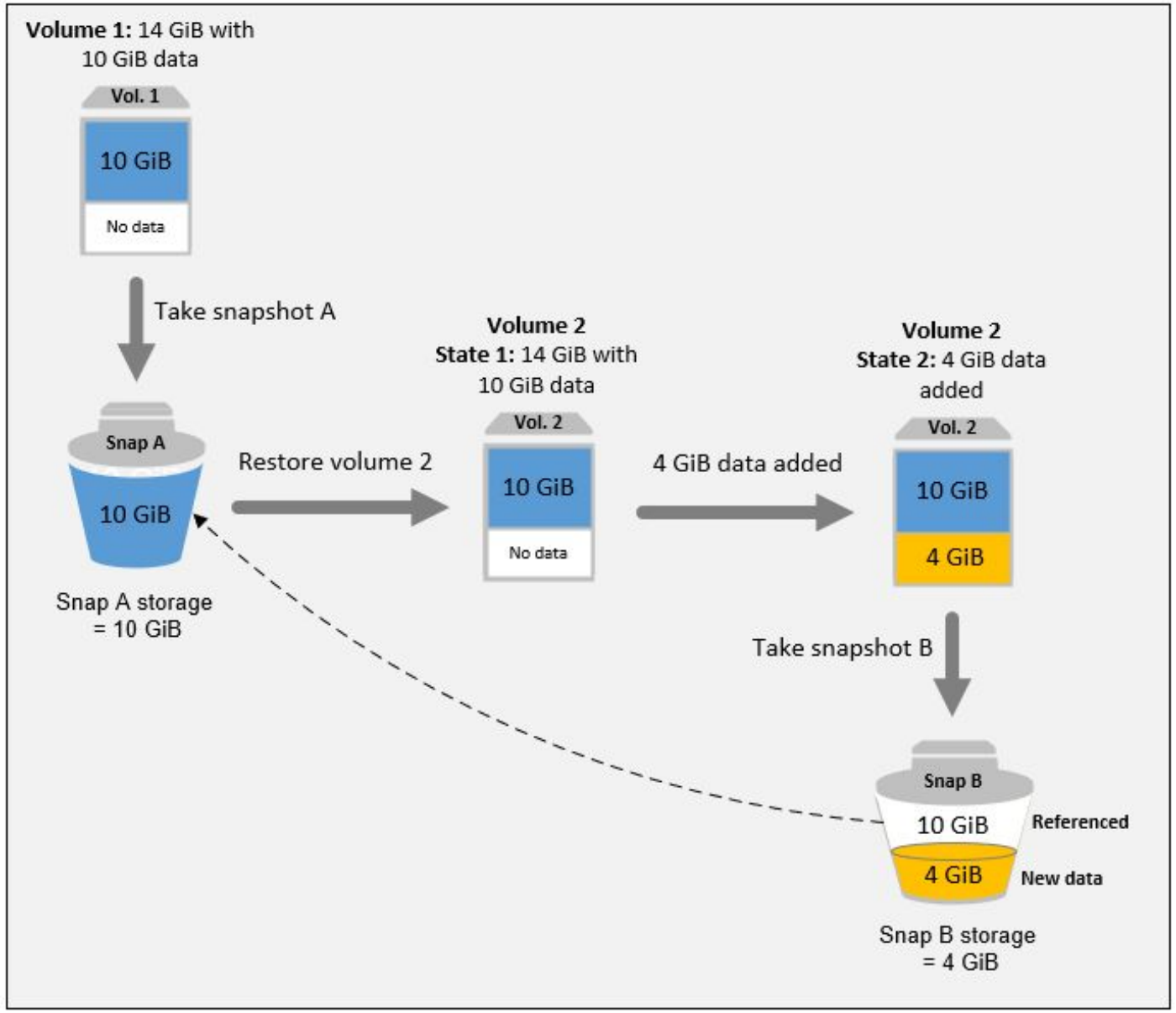
Elastic Block Service

Snapshot Architecture



Elastic Block Service

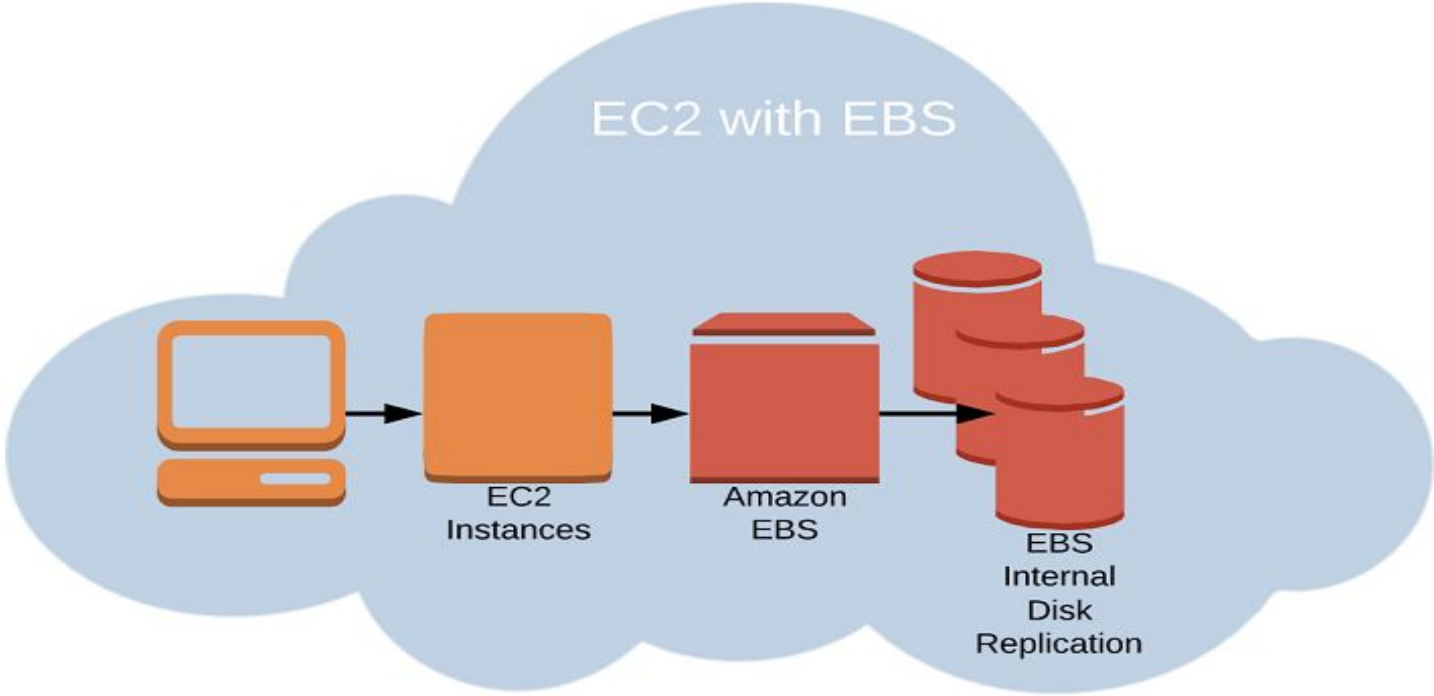
Backup Flow



Elastic Block Service

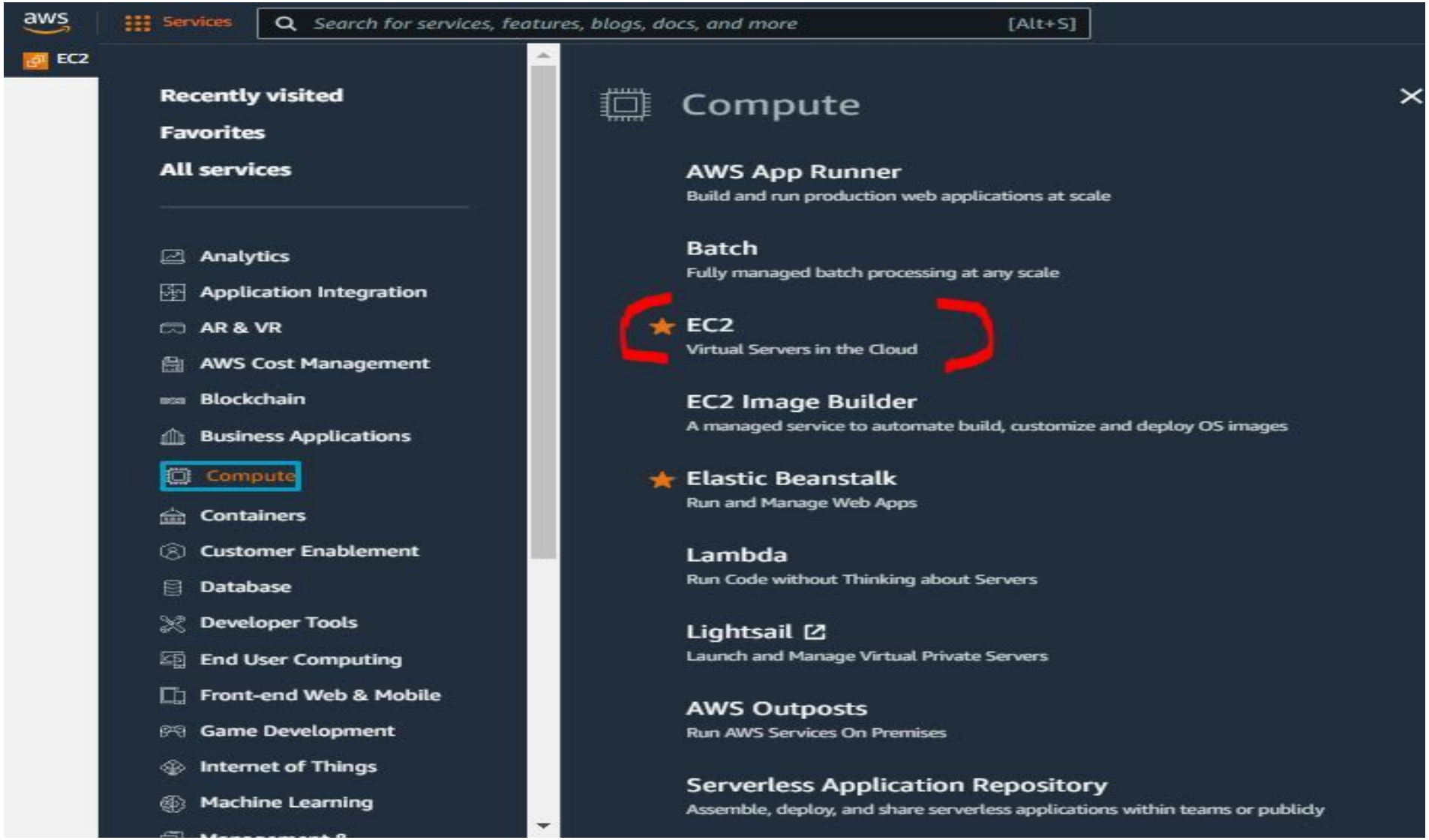
EBS Migration - EBS Volumes are locked to specific AZ. To migrate it to a different AZ or (Region) below steps to be followed:

1. Snapshot the Volume
2. Copy the volume to different region (Optional)
3. Create a Volume from the snapshot in the availability zone of your choice.



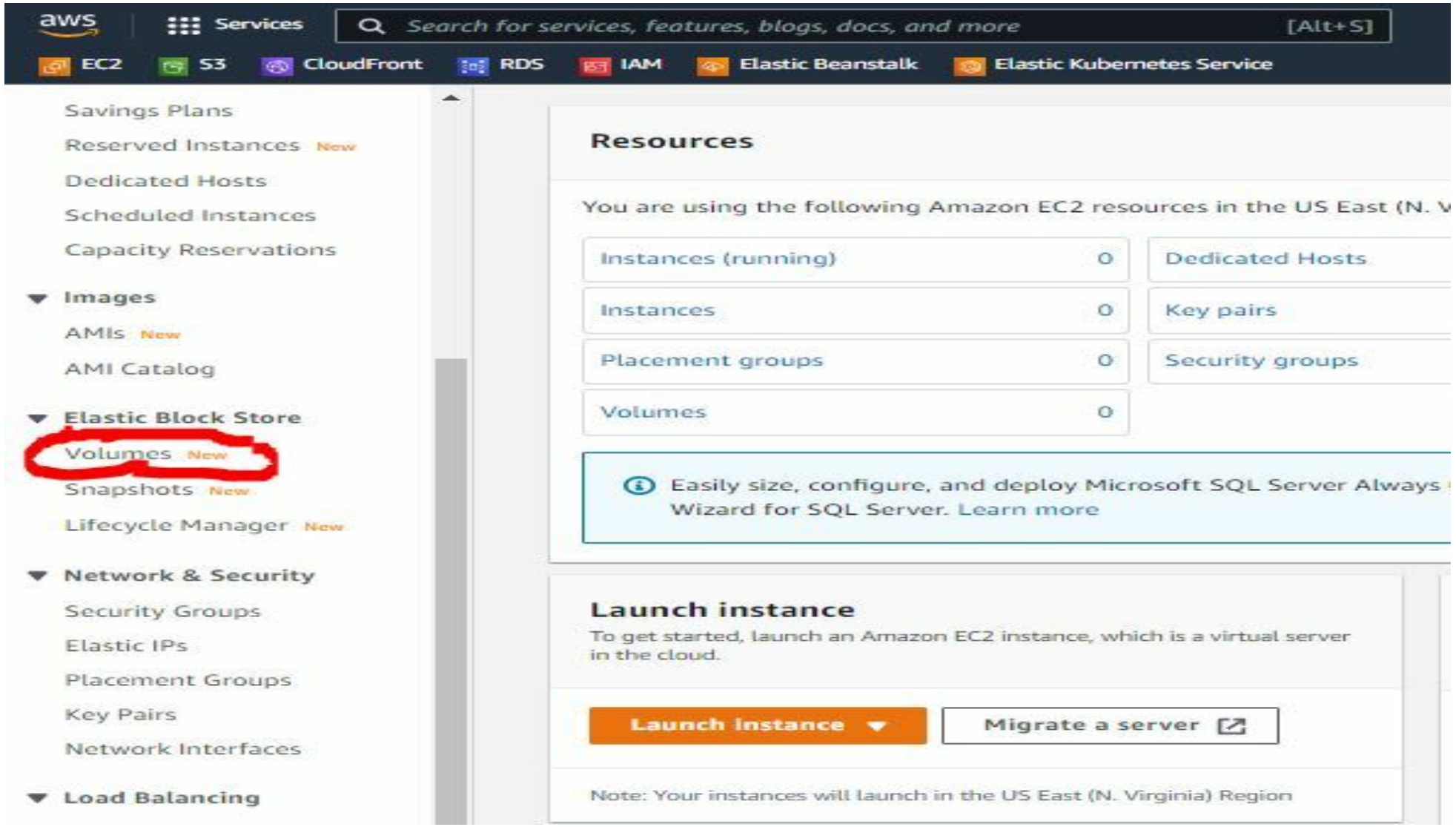
How to create EBS Volume

- **Step 1 -** Login to your AWS console, on the top right of the screen, navigate to **Service** > **Compute** > **EC2**



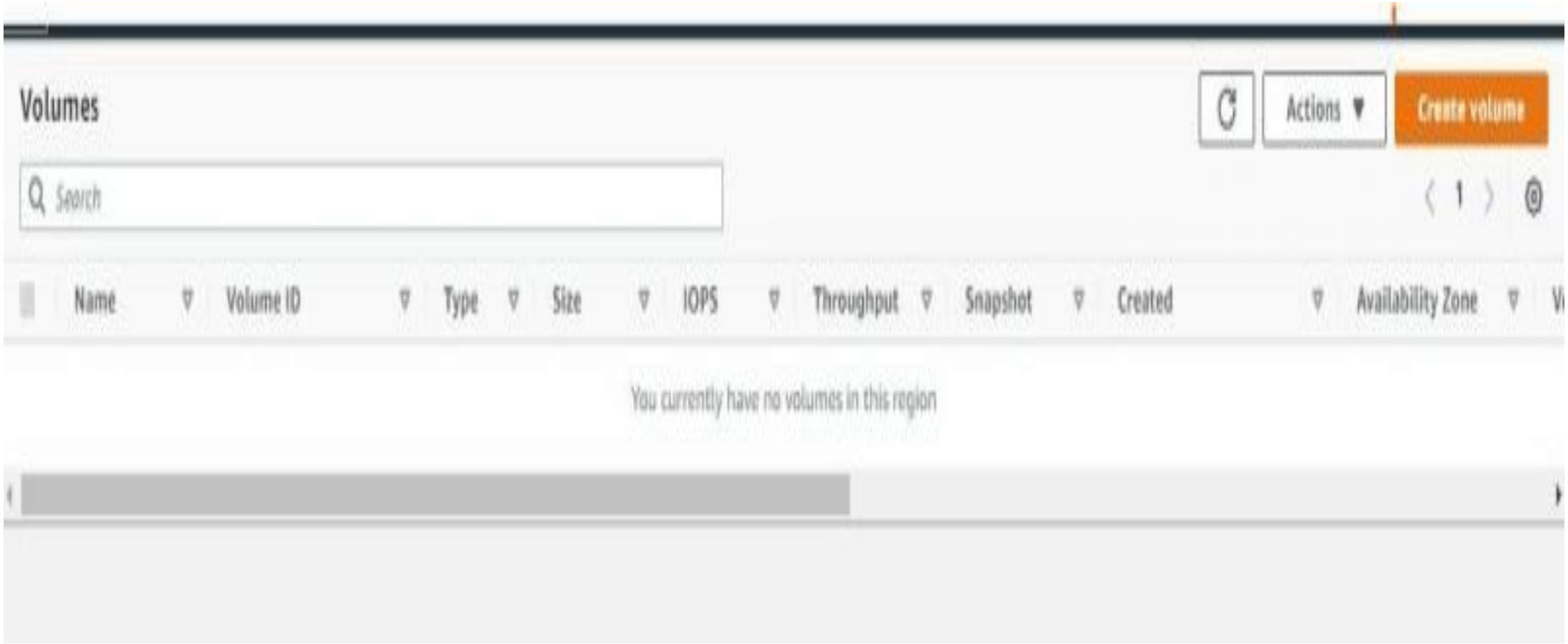
How to create EBS Volume

Step 2 - You are on the EC2 Dashboard, on the left navigation pane, scroll down to Elastic Block Store , *click on Volumes*.



How to create EBS Volume

Step 3 - On the volumes dashboard, click on **Create Volume**



How to create EBS Volume

Step 4 Volume Settings

On Create Volumes, for volume setting use these values:

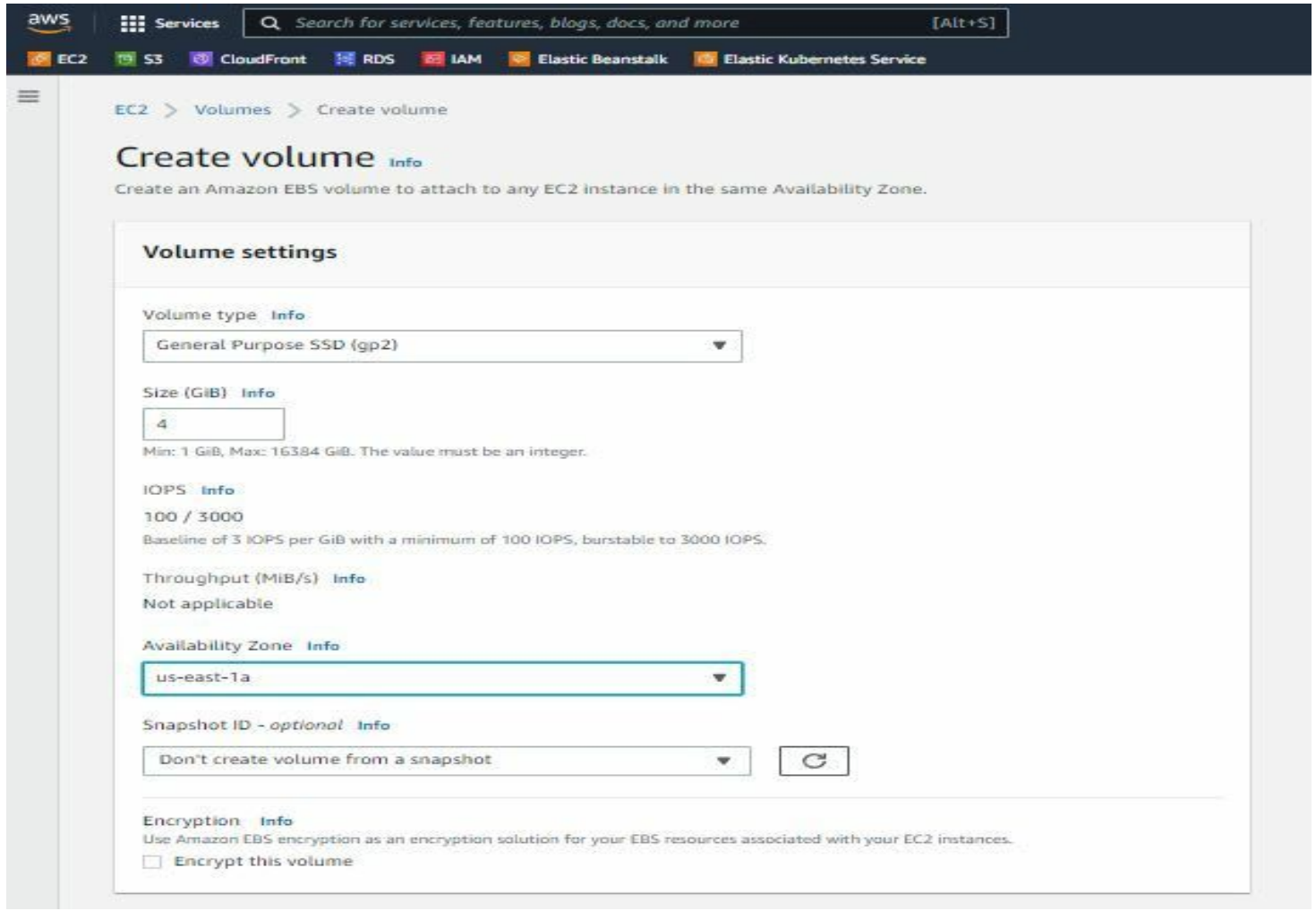
Volume type: General Purpose SSD (gp2)

Size GB: 4

Availability Zone: your choice,

but I would use

Us-east-1 for this exercise.



How to create EBS Volume

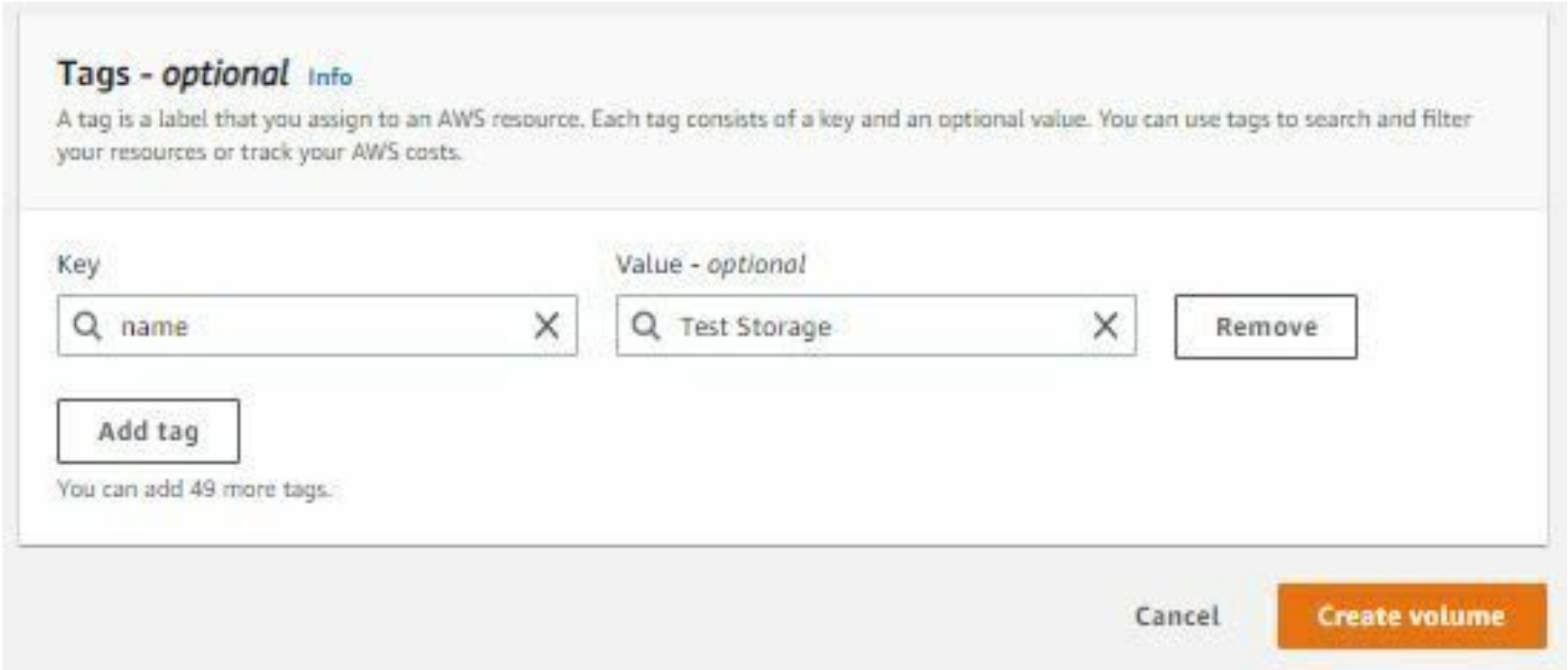
Step 5 Tags

Use these values:

Key: name

Value: Test Storage

Click on **Create Volume**

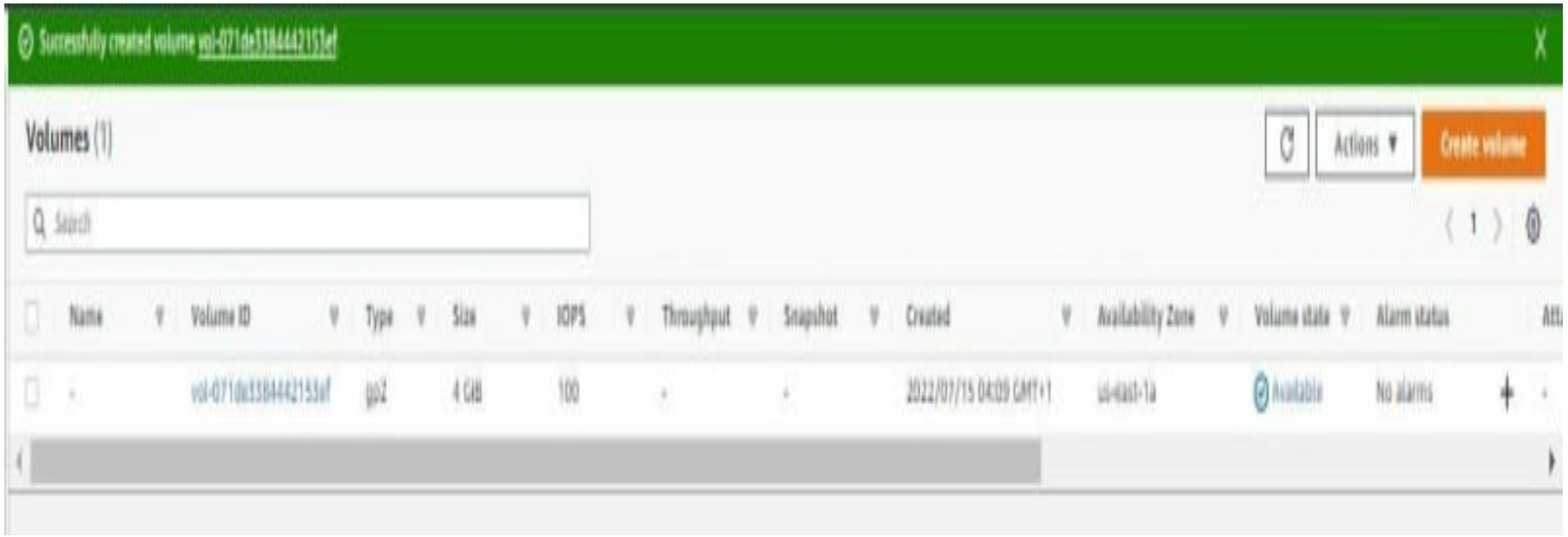


How to create EBS Volume

Step 6 Volume Dashboard

This dashboard displays a list of created volumes in this availability zone (us-east-1), with the volume state.

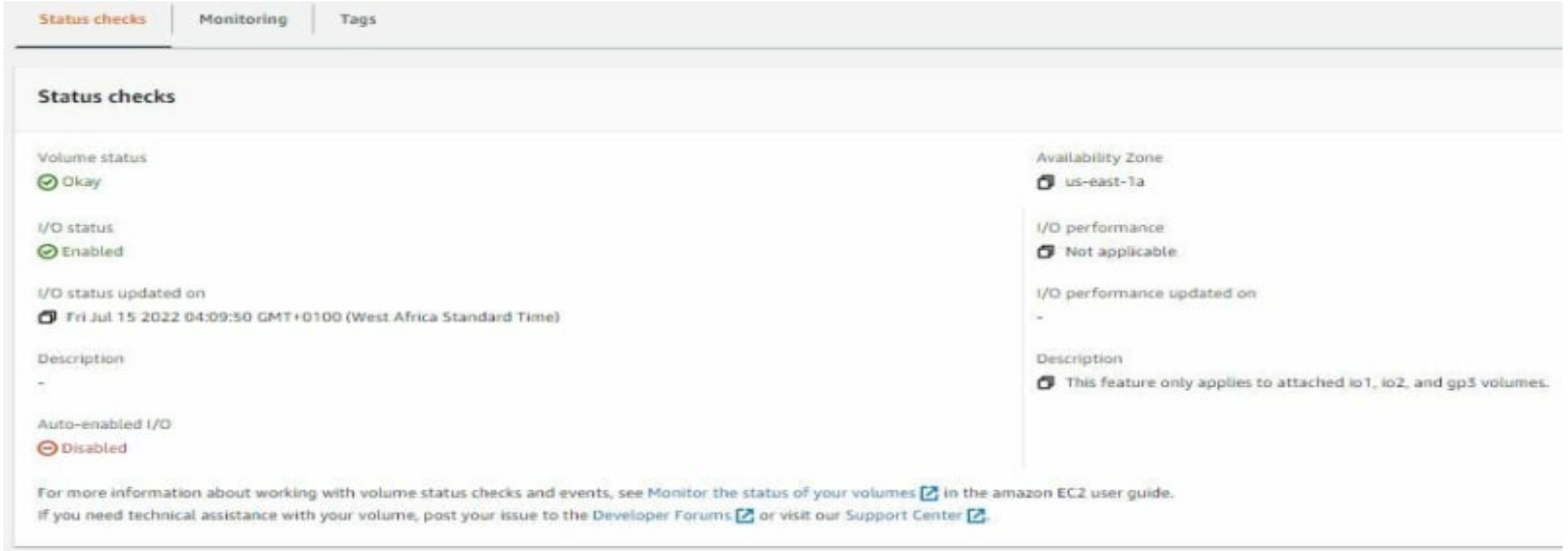
Click on the newly created volume ID



How to create EBS Volume

Step 7 Volume Status Checks

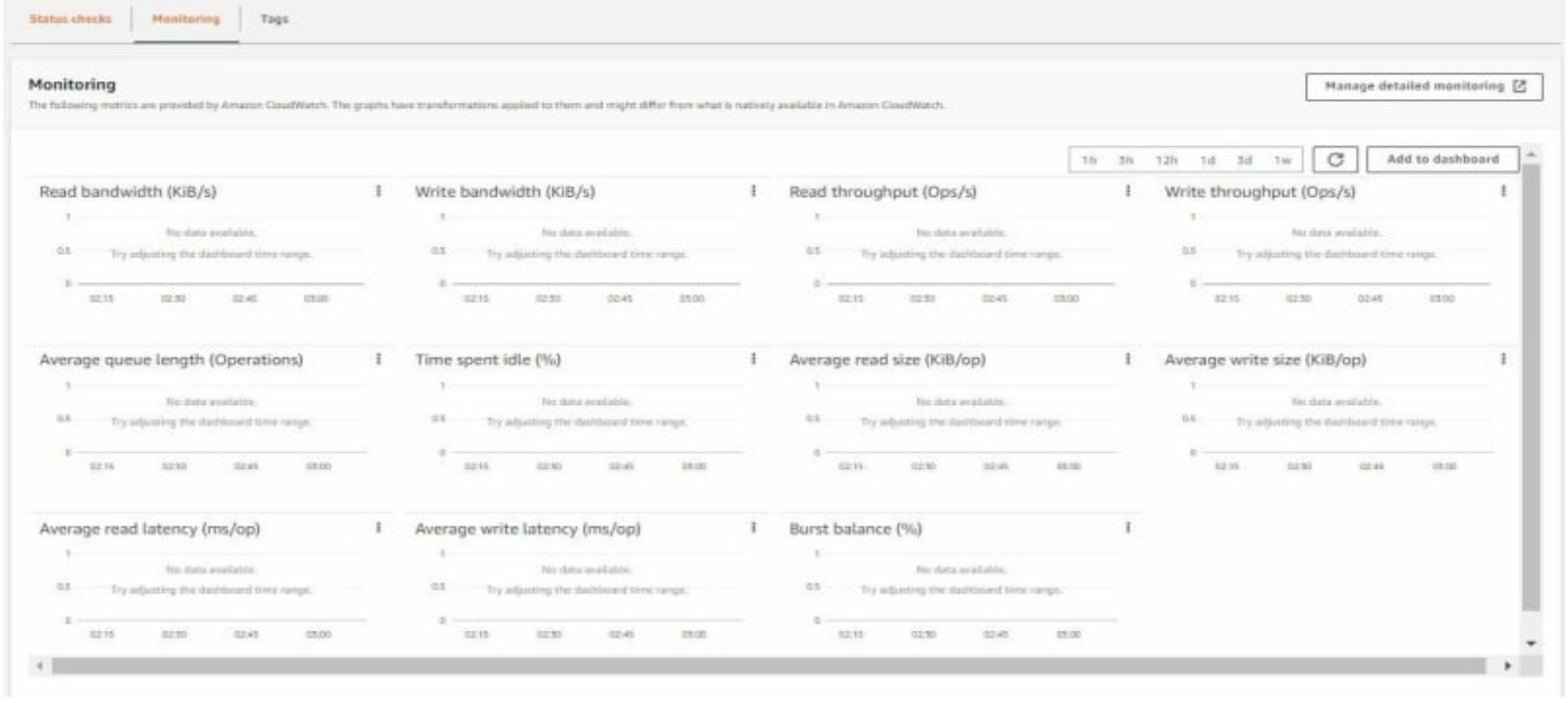
scroll down to status check, displayed is the Volume status, I/O status, Availability Zone, and some other relevant information.



How to create EBS Volume

Step 8 Monitoring

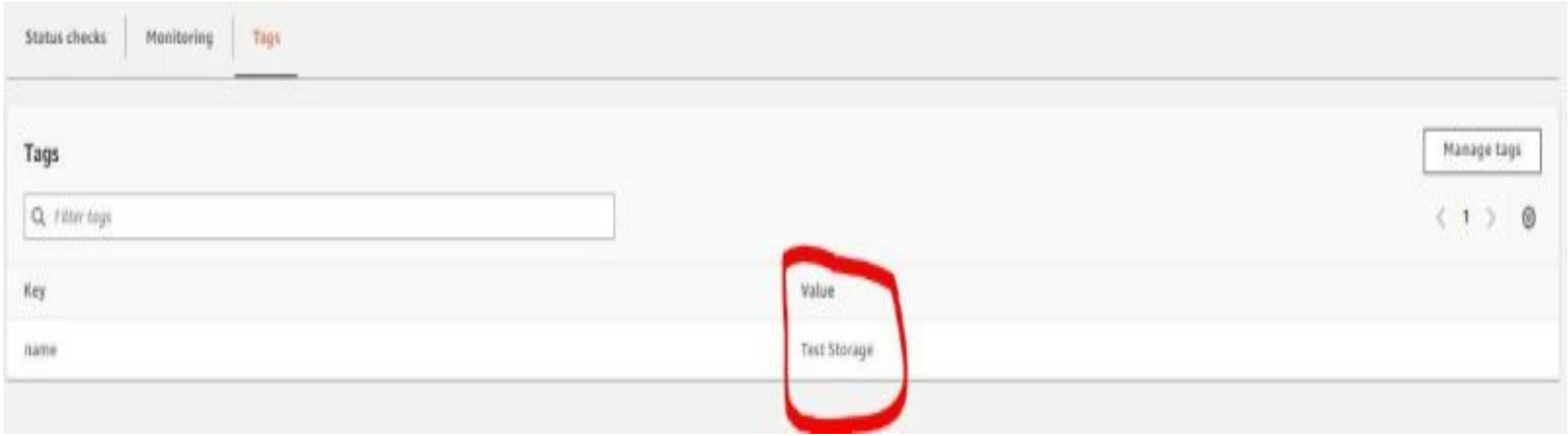
Click on monitoring, displayed is an extensive metrics, and applied graph transformations, provided, by Cloud Watch. The topic Cloud Watch will be covered in a future session



How to create EBS Volume

Step 9 Tags.

Displayed, Key: name | value : Test Storage



THE END

You have successfully created an Amazon EBS Volume.

S3 GLacier

Amazon S3 Glacier is a secure, ultra-low-cost cloud storage class designed for **data archiving** and long-term backup. While it offers the same 99.999999999% (11 9s) durability as standard S3, it is optimized for "cold" data that is rarely accessed, trading immediate retrieval speed for significantly lower storage costs (starting at **$1 per terabyte/month**).

- Glacier data repository is known as **“Vault”**
- Any data uploaded in vault is known as **“archive”**
- You can only delete a vault which is empty.

Amazon Glacier

An economical storage solution to store data that would remain forever, but, rarely accessed.

Ideal choice for data backup and archiving

Provides data security of the highest level

Offers flexibility in storing and retrieving data

AWS bills you only for the used data or storage, and current least price for storing data in Amazon Glacier is $0.007 per gigabyte per month.

S3 GLacier

The Retrieval Trade-off

The core mechanic of Glacier is the "**Retrieval Time**." Unlike standard storage where files open instantly, Glacier (Flexible & Deep) requires you to initiate a "Restore Job" to thaw the data before you can download it.

S3 GLacier

Key Benefits

1. Massive Cost Reduction

Glacier Deep Archive is approximately **99% cheaper** than S3 Standard. It is designed to replace physical magnetic tape libraries, allowing you to store petabytes of data for a fraction of the cost of on-premise maintenance.

2. Unmatched Compliance (Vault Lock)

For industries like healthcare (HIPAA) and finance (SEC Rule 17a-4), Glacier offers **Vault Lock**. This feature enforces a **WORM** (Write Once, Read Many) policy, legally ensuring that records cannot be deleted or altered by anyone (including administrators) until a specific retention period expires.

3. Automated Lifecycle Management

You generally do not upload directly to Glacier. Instead, you set **S3 Lifecycle Policies** that automatically move data from "Hot" tiers (S3 Standard) to "Cold" tiers (Glacier) as it ages, automating cost savings without manual intervention.

S3 GLacier

Common use cases

- **Long-Term Backup & Disaster Recovery:** Serving as a cost-effective, secondary copy of critical data that is only needed in emergencies or for planned monthly/quarterly restores.
- **Media Asset Workflows:** Storing massive libraries of raw video footage, news archives, and high-resolution images that are kept indefinitely but rarely viewed.
- **Healthcare Records Preservation:** Maintaining decades of patient history, genomic sequences, and medical imaging (like X-rays or MRIs) at a fraction of on-premises costs.
- **Scientific Data Storage:** Archiving vast amounts of research data, satellite imagery, or historical datasets for future analysis or machine learning training.
- **Magnetic Tape Replacement:** Replacing expensive, high-maintenance on-premises tape libraries with a more durable and easily managed cloud alternative.

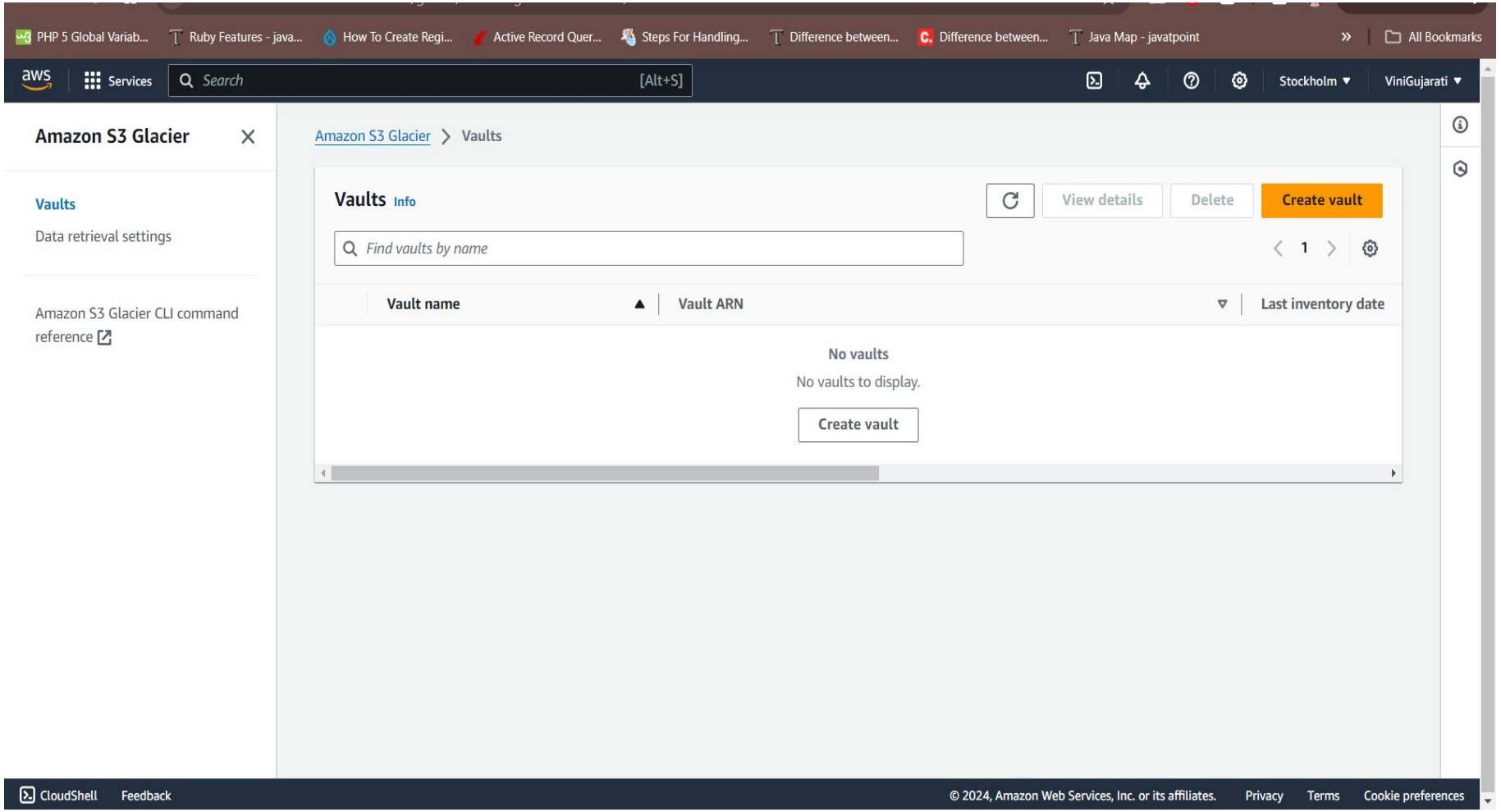
S3 GLacier

Choosing the Right Glacier Tier

Depending on your specific use case, AWS offers three sub-types:

Tier	Best Use Case	Retrieval Time
Instant Retrieval	"Active" archives like medical images or news media needed immediately.	Milliseconds
Flexible Retrieval	Standard backups and disaster recovery where a few hours' wait is fine.	1 min – 12 hours
Deep Archive	"Deep freeze" for compliance or records kept for 7–10+ years.	12 – 48 hours

S3 GLacier


PHP 5 Global Variab...
Ruby Features - java...
How To Create Regi...
Active Record Quer...
Steps For Handling...
Difference between...
Difference between...
Java Map - javatpoint
All Bookmarks
aws
Services
Search
[Alt+S]
Stockholm
ViniGujarati
Amazon S3 Glacier
Vaults
Data retrieval settings
Amazon S3 Glacier CLI command reference
Amazon S3 Glacier > Vaults
Vaults Info
View details
Delete
Create vault
Find vaults by name
1
Vault name
Vault ARN
Last inventory date
No vaults
No vaults to display.
Create vault
CloudShell
Feedback
© 2024, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

Database Services

AWS offers over 15 purpose-built database services, each designed for specific performance, scale, and data model requirements. These are fully managed, meaning AWS handles administrative tasks like hardware provisioning, patching, and backups.

AWS database services mainly fall into two categories: **Relational** and **non-relational.**

- **Amazon RDS –** Aurora, Oracle, PostgreSQL, MySQL, MariaDB, SQL Server
- **DynamoDB**
- **ElasticCache** – Memcached, Redis
- **Redshift**

Database Services

AWS offers over 15 purpose-built database services, each designed for specific performance, scale, and data model requirements. These are fully managed, meaning AWS handles administrative tasks like hardware provisioning, patching, and backups.

AWS database services mainly fall into two categories: **Relational** and **non-relational.**

- **Amazon RDS –** Aurora, Oracle, PostgreSQL, MySQL, MariaDB, SQL Server
- **DynamoDB**
- **ElasticCache** – Memcached, Redis
- **Redshift**

Database Services

Introduction:

- Cloud database services allow you to set-up and operate relational or non-relational databases in the cloud.
- The benefit of using cloud database services is that it relieves the application developers from the time consuming database administration tasks.
- Popular relational databases provided by various cloud service providers include MySQL, oracle, SQL server etc.
- The non-relational (No-SQL) databases provided by cloud service providers are mostly proprietary solutions.
- No-SQL databases are usually fully-managed.

Database Services

Database Services

1. Relational Databases (SQL)

Best for structured data with complex relationships and transactional consistency (ACID compliance).

- **Amazon RDS:** Supports six familiar engines: MySQL, PostgreSQL, MariaDB, Oracle, SQL Server, and IBM Db2.
- **Amazon Aurora:** A cloud-native relational database that is up to 5x faster than standard MySQL and 3x faster than standard PostgreSQL.
- **Use Cases:** ERP/CRM systems, financial applications, and traditional web apps.

Database Services

Database Services

2. Non-Relational Databases (NoSQL)

Designed for extreme scale and consistent single-digit millisecond latency.

- **Amazon DynamoDB:** A serverless, non-relational database that can handle more than 10 trillion requests per day.
- **Use Cases:** Shopping carts, real-time bidding, gaming leaderboards, and high-traffic web sessions.

Database Services

Database Services

3. In-Memory Databases

Used for ultra-fast data retrieval with microsecond latency by storing data in RAM.

- **Amazon ElastiCache:** A managed caching service compatible with Redis and Memcached.
- **Amazon MemoryDB:** A Redis-compatible database that adds durability to in-memory performance.
- **Use Cases:** Real-time analytics, session management, and caching layers.

Database Services

RDS

- Manages relational database
- Handles structured & tabular data

Dynamo

- NoSQL database
- Document and key-value store
- Handles unstructured data

ElastiCache

- In-Memory Cache
- Improves the performance of any web application

Redshift

- Data Warehouse
- It is used for data analysis and reporting

Aurora

- MySQL-compatible relational database with 5X performance

What is Relational Database Services?

Amazon Relational Database Service (Amazon RDS)

- It makes it simple to set up, scale, and run relational databases on the cloud. It offers scalable and cost-effective capacity while performing database administration responsibilities, allowing you to focus on your business and applications. It supports six database engines: **Amazon Aurora, MariaDB, MySQL, PostgreSQL, Microsoft SQL Server, and Oracle.**
- It provides a **reliable and stable solution** for real-time data management, and you always have analysis-ready data at your selected location. It enables you to spend time on critical business objectives .
- **Amazon Relational Database Service (Amazon RDS)** makes it easy to spin up a database in the AWS cloud **in just a few minutes.**
- RDS is the most commonly used and managed database service that automates all the time-consuming administration tasks such as **provisioning, setup, patching, and backups.**

What is Relational Database Services?

Amazon RDS provides six different relational database options:

- Amazon Aurora
- Oracle Database
- PostgreSQL
- MySQL
- MariaDB
- Microsoft SQL Server

How Relational Database Services Work?

Working:

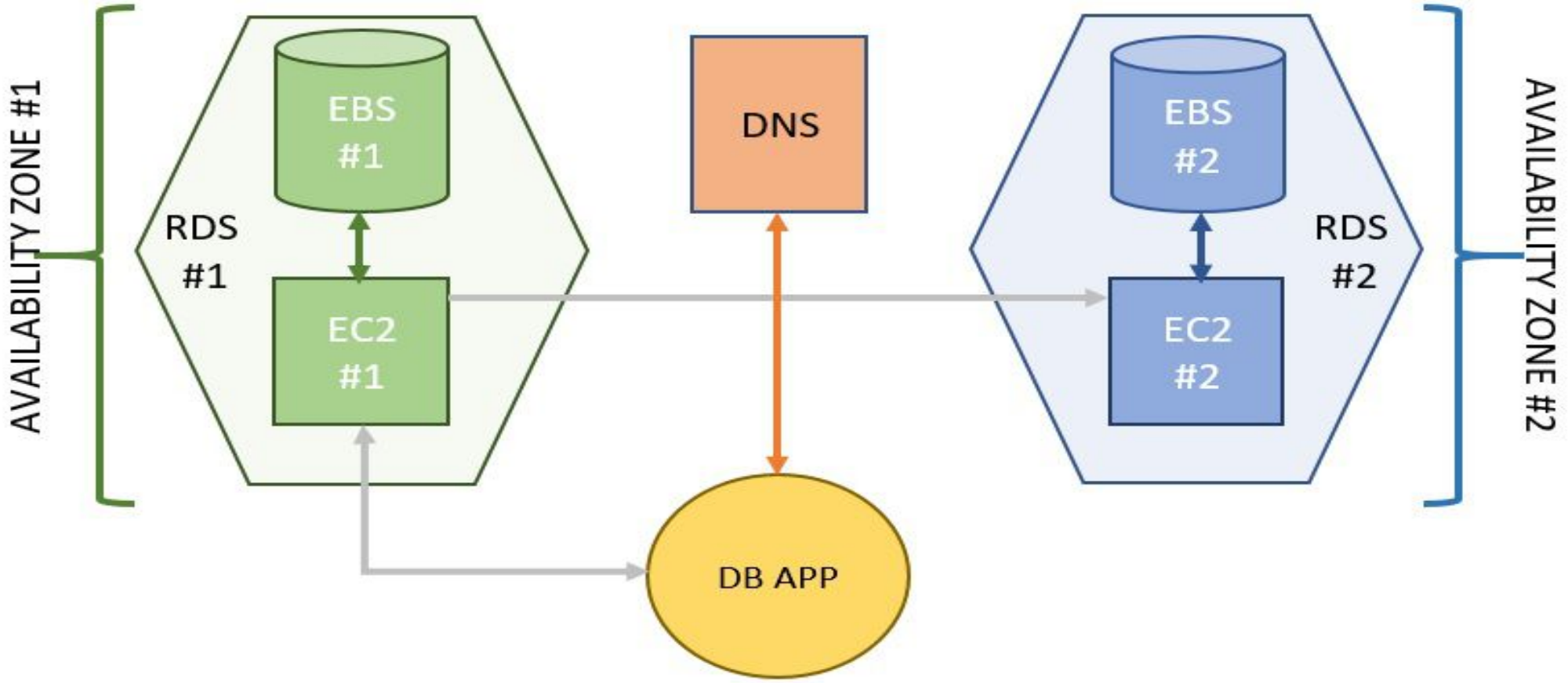
DB Instance: The basic building block is a DB instance, which is an isolated database environment in the cloud. You choose the "instance class" (CPU and RAM) and the "storage type" based on your performance needs.

Control Plane vs. Data Plane: The **Control Plane** handles administrative tasks like provisioning, patching, and backups. The **Data Plane** is where your actual database engine runs, processing SQL queries and managing data operations.

Database Engines: RDS supports multiple familiar engines, including MySQL, PostgreSQL, MariaDB, Oracle, and Microsoft SQL Server, as well as cloud-native options like **Amazon Aurora**.

How Relational Database Services Work?

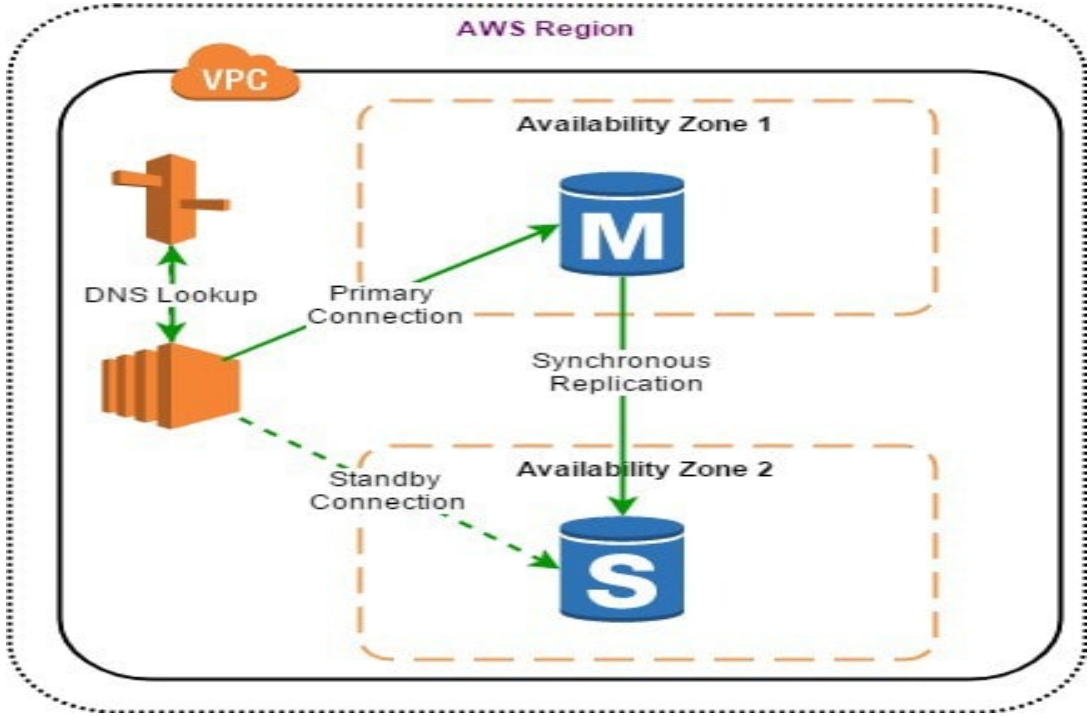
RDS (Relational Database Service) includes and is built on top of **EC2** and **EBS**—but AWS manages them for you so you don't have to deal with them directly.



How Relational Database Services Work?

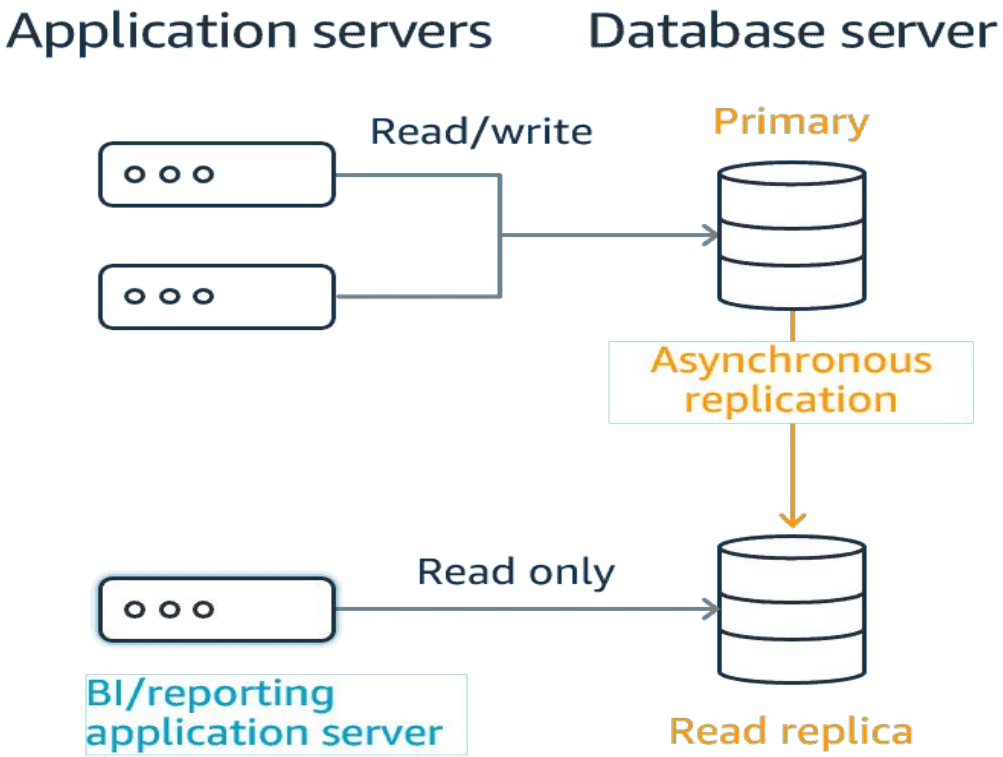
Amazon RDS has two main features:

1. Multi-AZ Deployments - Amazon RDS Multi-Az deployments provide high availability and failover support. Amazon RDS allows you to have **multiple copies of your database** in **multiple availability zones**. Amazon RDS creates a synchronous standby replica of your DB instance in another Availability Zone and automatically switches or failover to a standby replica in another Availability Zone in case of a planned or unplanned outage if you have enabled Multi-AZ



How Relational Database Services Work?

2. Read Replicas - Amazon RDS Read replicas offer the advantage of **having a read-only copy of your database in the same or a separate area**, **minimizing latency.** When the source database instance is changed, the modifications are asynchronously replicated to the read replica. Read replicas are most commonly employed for read-intensive database workloads.



Benefits of RDS

- **Flexible -** There are various RDS pricing models available, such as on-demand and reserved instances. So, whatever your specific requirements are, there is sufficient flexibility to meet your use case.
- **Secure -** In terms of security and running an on-premises or hybrid solution, RDS can also be run in the Amazon Virtual Cloud and is encrypted in transit and at rest.
- **Adaptable -** RDS databases can be scaled at the touch of a button and are highly available, with the option of replicating data to an instance in a different zone – so you're covered if a server fails.
- **Simple administration -** It is simple to set up an RDS database. RDS's infrastructure is automatically provisioned as a cloud-based service, so you don't have to set up the database software. On the other hand, RDS is available with on-premises and hybrid solutions. Once your RDS database is up and running, you can forget about admin tasks like software patching and backups because AWS will handle everything for you.
- **A market leader -** RDS is a popular database, with prestigious companies such as Netflix, Expedia, and global consumer goods brand Unilever as customers. AWS was the first company to bring cloud computing into the spotlight, and they are still the industry's go-to source for innovation.

Non Relational DB (Amazon Dynamo DB)

AWS DynamoDB is a serverless, non-relational database that can handle more than 10 trillion requests per day. It is designed for extreme scale and consistent single-digit millisecond latency.

Use Cases: Shopping carts, real-time bidding, gaming leaderboards, and high-traffic web sessions.

Key Characteristics

- **Serverless & Fully Managed:** You don't have to provision servers, patch software, or manage physical infrastructure. AWS handles the "heavy lifting," including hardware provisioning, setup, and configuration.
- **Flexible Data Model:** It supports both **key-value** and **document** data structures (using JSON). Unlike traditional relational databases, DynamoDB is schemaless, meaning different items in the same table can have different attributes.
- **Automatic Scaling:** It can scale up or down automatically to meet application demand. In **on-demand mode**, you pay only for the requests your application performs, with no capacity planning required.
- **High Availability & Durability:** Data is automatically replicated across three AWS Availability Zones within a region to ensure 99.999% availability and data protection.

Non Relational DB (Amazon Dynamo DB)

Core Components

- **Tables:** The top-level collection of data (similar to a table in SQL).
- **Items:** Individual records within a table, uniquely identified by a primary key (similar to a row).
- **Attributes:** Fundamental data elements within an item (similar to a column), which can be scalars, sets, or document types.
- **Primary Key:** Must be unique for every item. It can be a simple **Partition Key** or a composite **Partition Key + Sort Key**.

Network Services

In **cloud computing**, network services are the virtualized infrastructure and protocols that enable communication, data transfer, and security between users, applications, and cloud-hosted resources. Unlike traditional networking that relies on physical hardware like on-site routers and switches, these services are software-defined and typically managed through a centralized cloud dashboard.

Core Network Components

- **Virtual Private Cloud (VPC):** A logically isolated section of a public cloud where you can launch resources in a virtual network you define, including your own IP address ranges and subnets.
- **Subnets:** Segments of a VPC used to organize and isolate resources, such as keeping databases in a private subnet and web servers in a public one.
- **Load Balancers:** Tools that distribute incoming network traffic across multiple servers to ensure high availability and prevent any single resource from being overwhelmed.
- **Cloud Gateways:** Virtual routers that allow traffic to flow between your VPC and the internet (Internet Gateway) or between your VPC and an on-premises network.
- **Content Delivery Network (CDN):** A distributed network of servers that caches content closer to end-users to reduce latency and speed up delivery.

Network Services

Connectivity and Security Services

- **Virtual Private Network (VPN):** Provides secure, encrypted communication between remote users or on-premises data centers and your cloud resources.
- **Direct Connect:** A dedicated, private network connection between an organization's on-premises infrastructure and the cloud provider, bypassing the public internet for better performance.
- **Cloud Firewalls & Security Groups:** Virtual firewalls that control inbound and outbound traffic based on predefined security rules at the network or instance level.
- **Domain Name System (DNS):** A managed service that translates domain names into IP addresses, often with intelligent routing to the nearest available server.

Advanced Service Models

- **Network as a Service (NaaS):** A model where customers rent networking services—such as virtual switches, routers, and firewalls—directly from a cloud vendor, eliminating the need to maintain any physical networking hardware.
- **Secure Access Service Edge (SASE):** An architecture that combines wide-area network (WAN) capabilities with cloud-native security functions (like firewalls and secure web gateways) into a single integrated service.

Network Services

Key Benefits

- **Scalability:** Resources can be scaled up or down instantly to meet demand without purchasing new hardware.

- **Cost Efficiency:** Most services follow a pay-per-use model, converting capital expenditures (CapEx) into predictable operational expenses (OpEx).

- **Reliability:** Cloud providers use globally distributed data centers to offer high availability and built-in disaster recovery.

Security Services

Cloud security services protect your data, applications, and infrastructure through a **shared responsibility model**. While cloud providers (like AWS, **Azure**, or **Google Cloud**) secure the underlying "cloud," you are responsible for securing what you put "in" it.

Core Security Categories

- **Identity and Access Management (IAM):** Acts as a "digital bouncer" to ensure only authorized users access specific resources. Key tools include **Multi-Factor Authentication (MFA)** and **Single Sign-On (SSO)**.
- **Data Security & Encryption:** Scrambles data to make it unreadable without a key. This applies to **data at rest** (stored) and **data in transit** (moving between systems).
- **Network Security:** Uses virtual barriers like **Cloud Firewalls**, **Web Application Firewalls (WAF)**, and **Virtual Private Clouds (VPC)** to block malicious traffic and isolate resources.
- **Compliance and Governance:** Automated tools like **Cloud Security Posture Management (CSPM)** monitor your environment for misconfigurations and ensure you meet legal standards like **GDPR**, **HIPAA**, or **PCI DSS**.
- **DDoS Protection:** Absorbs massive floods of traffic to prevent service outages.